arXiv:2605.19376v4 [cs.AI] 15 Oct 2026

Generative Recursive Reasoning

Junyeob Baek[1†*] **Mingyu Jo**[1†*] **Minsu Kim**[1,2]

Mengye Ren[3] **Yoshua Bengio**[2,4] **Sungjin Ahn**[1,3†]

[1]KAIST [2]Mila – Québec AI Institute
[3]New York University [4]Université de Montréal

Abstract

How should future neural reasoning systems implement extended computation? Recursive Reasoning Models (RRMs) offer a promising alternative to autoregressive sequence extension by performing iterative latent-state refinement with shared transition functions. Yet existing RRMs are largely deterministic, following a single latent trajectory and converging to a single prediction. We introduce *Generative Recursive reAsoning Models (GRAM)*, a framework that turns recursive latent reasoning into probabilistic multi-trajectory computation. GRAM models reasoning as a stochastic latent trajectory, enabling multiple hypotheses, alternative solution strategies, and inference-time scaling through both recursive depth and parallel trajectory sampling. This yields a latent-variable generative model supporting conditional reasoning via $p_\theta(y \mid x)$ and, with fixed or absent inputs, unconditional generation via $p_\theta(x)$. Trained with amortized variational inference, GRAM improves over deterministic recurrent and recursive baselines on structured reasoning and multi-solution constraint satisfaction tasks, while demonstrating an unconditional generation capability. https://ahn-ml.github.io/gram-website

1 Introduction

A central question for future neural reasoning systems is how extended computation should be implemented. Large autoregressive models typically scale reasoning by extending a sequence-generation process, whether intermediate computation is expressed explicitly as chain-of-thought tokens or implicitly in hidden or latent representations [1–6]. A complementary direction is explored by Recursive Reasoning Models (RRMs), which use repeated computation to refine a persistent latent state rather than to append new elements to an output or reasoning sequence [7–9]. This approach is appealing because it decouples reasoning depth from both parameter scale and output length: a compact model can perform many steps of internal computation by repeatedly applying shared transition functions.

Recent recursive reasoning models such as HRM [8] and TRM [9] provide early evidence for the potential of this approach in structured reasoning. Rather than producing a solution in a single feedforward pass, they perform extended computation through iterative latent-state refinement, deep supervision across refinement steps, and reasoning-oriented recurrent designs such as hierarchical latent dynamics. These features make them well suited to problems requiring constraint propagation, state tracking, iterative correction, and multi-step inference. More broadly, they build on a principle also explored in recurrent Transformer architectures such as Universal Transformers [10] and Looped Transformers [7]: shared Transformer blocks can be repeatedly applied to increase computational

*Equal contribution
†Correspondence to: Sungjin Ahn (sungjin.ahn@kaist.ac.kr)

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

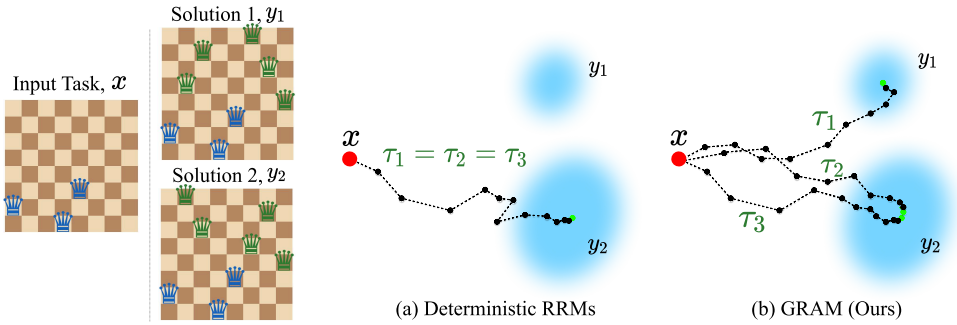


Figure 1: Comparison of Latent Reasoning Trajectories. Left: N-Queens Example with two valid solutions. Right: Given three independent runs for latent reasoning (τ_1, τ_2, τ_3): (a) Prior RRMs (e.g. HRM, TRM) are deterministic—all runs collapse to an identical trajectory, converging to a single solution and failing to explore alternatives, while (b) GRAM explores diverse trajectories, producing diverse trajectories that reach multiple valid solutions y_1 and y_2, while naturally enabling parallel inference-time scaling.

depth without increasing parameter count. Together, these models suggest that reasoning capability can emerge not only from scaling model size or generating longer traces, but also from the organization of computation itself.

While recurrent latent-state refinement provides an appealing mechanism for efficiently increasing reasoning depth, depth alone is not sufficient for many reasoning problems. A capable reasoning system should also be able to maintain uncertainty, consider alternative hypotheses, and explore multiple possible solution strategies [11, 12]. This is especially important in settings where ambiguity or multiple valid solutions are intrinsic, and more generally in problems where a single refinement path may become trapped in a suboptimal reasoning trajectory. In this sense, future RRMs should be not only deep, in the sense of repeated refinement, but also wide, in the sense of maintaining and exploring multiple latent trajectories in parallel.

Existing RRMs [7–10], however, remain fundamentally deterministic: given the same input and initialization, they follow a single latent trajectory and converge to a single prediction. This deterministic recursion collapses the space of plausible reasoning paths into a single attractor, leaving probabilistic multi-hypothesis latent reasoning largely unexplored within the RRM paradigm. This motivates the central question of our work: *can recursive latent computation support probabilistic, generative, multi-hypothesis reasoning while preserving the efficiency of compact recurrent models?*

In this paper, we propose Generative Recursive reAsoning Models (GRAM), a framework that turns recursive latent reasoning into probabilistic multi-trajectory computation. GRAM treats the reasoning process itself as a stochastic latent trajectory: at each recursion step, the model samples a transition conditioned on the input and the current reasoning state, rather than deterministically updating to a single next state. Repeating this process defines a distribution over possible reasoning trajectories, allowing the model to maintain multiple hypotheses, explore alternative solution strategies, and scale inference not only by increasing recursive depth but also by sampling trajectories in parallel. From a probabilistic perspective, GRAM is a latent-variable generative model. It models $p_\theta(y \mid x)$ by marginalizing over latent reasoning trajectories, while the same recursive process can also define an unconditional generative model $p_\theta(x)$ when the input is fixed or absent.

We evaluate GRAM on controlled reasoning and generation tasks that serve as probes of the architectural properties targeted by our formulation, namely recursive refinement, stochastic exploration, multi-solution coverage, and inference-time scaling. Sudoku-Extreme [8] and ARC-AGI [13, 14] test structured reasoning under hard constraints and abstract transformations; N-Queens and Graph Coloring evaluate multi-solution reasoning; and Sudoku and binarized MNIST [15] generation from empty inputs investigates the potential of the same recursive process as an unconditional generator. To isolate what stochastic recursion adds, we compare GRAM with the most relevant deterministic recurrent and recursive latent reasoning baselines, including Looped Transformers, HRM, and TRM, rather than frontier-scale general-purpose LLMs whose training data, inference budgets, and external scaffolding are not directly comparable.

Our main contribution is to establish probabilistic multi-trajectory recursion as a design principle for future recurrent and recursive reasoning architectures. Concretely, we make three contributions. First, we formulate recursive reasoning as a latent-variable generative process, where solutions are

obtained by marginalizing over stochastic reasoning trajectories. Second, we introduce width-based inference-time scaling, enabling inference to scale not only with recursive depth but also with the number of sampled latent trajectories. Third, we provide empirical evidence that this formulation yields the intended architectural advantages over deterministic recurrent and recursive baselines, improving structured reasoning, multi-solution constraint satisfaction, and unconditional generation.

2 Generative Recursive Reasoning Models

In this section, we introduce Generative Recursive reAsoning Models (GRAM), an instantiation of probabilistic recursive reasoning. We describe the architecture in Section 2.1 and the training procedure in Section 2.2, with an architecture schematic shown in Figure 2; pseudocode for both training and inference is given in Appendix A.1.

2.1 Architecture

Overview. GRAM models the conditional distribution $p_\theta(y \mid x)$ by marginalizing over stochastic latent reasoning trajectories. Given an input x, GRAM first computes an embedding

$$e_x = f_{\text{enc}}(x; \theta), \tag{1}$$

which is reused throughout the entire recursive computation. Starting from a fixed initial latent state z_0, the model evolves the latent state through learned stochastic transitions. The recursive computation is organized into two nested levels: inner and outer loops.

At the inner level, a *latent transition* samples a new latent state conditioned on the previous latent state and the input embedding,

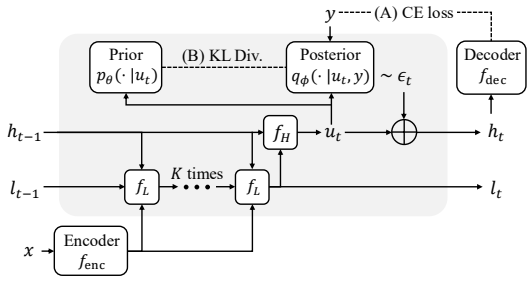


Figure 2: GRAM Architecture. A single stochastic latent transition in the hierarchical instantiation $z = (h, l)$. After K low-level refinements via f_{L}, the high-level update f_{H} produces a deterministic proposal u_t, to which stochastic guidance ϵ_t is added: $h_t = u_t + \epsilon_t$.

$$z_t \sim p_\theta(z_t \mid z_{t-1}, e_x), \qquad t = 1, \ldots, T. \tag{2}$$

At the end of the T transitions, the decoder produces a prediction, $\hat{y} = \arg\max f_{\text{dec}}(z_T; \theta)$. We refer to the sequence of T transitions from the initial state z_0 to the final state z_T as a *supervision step*. A supervision step is the unit at which the decoder is invoked, and the training objective is applied, with gradients computed as described in Section 2.2.

At the outer level, N_{sup} supervision steps are applied recursively, with the final state of one supervision step serving as the initial state of the next, thereby forming the full recursive computation:

$$z_0^{(1)} \xrightarrow{T \text{ transitions}} z_T^{(1)} = z_0^{(2)} \xrightarrow{T \text{ transitions}} \cdots \xrightarrow{T \text{ transitions}} z_T^{(N_{\text{sup}})}, \tag{3}$$

where $z_t^{(n)}$ denotes the latent state at the t-th transition of the n-th supervision step, $z_0^{(1)}$ is the fixed initial state, and the terminal state of one supervision step serves as the initial state of the next ($z_0^{(n+1)} := z_T^{(n)}$). This abstract formulation can be instantiated with various recurrent Transformer backbones, including flat designs such as Universal Transformers and Looped Transformers [10, 7], as well as hierarchical designs such as HRM and TRM [8, 9].

Stochastic Latent Transitions. Unlike prior recursive reasoning models (RRMs) that update the latent state deterministically and follow a single fixed trajectory [8, 9], GRAM defines $p_\theta(z_t \mid z_{t-1}, e_x)$ as a stochastic transition, so that repeated computation induces a distribution over latent reasoning trajectories. Concretely, GRAM realizes this transition as a learned stochastic residual perturbation around a deterministic update. At each transition, the model first computes a deterministic update u_t from z_{t-1} and e_x, then samples a conditional perturbation from a state-dependent Gaussian, and adds it to u_t:

$$\epsilon_t \sim p_\theta(\epsilon_t \mid u_t) := \mathcal{N}\big(\mu_\theta(u_t),\, \sigma_\theta^2(u_t) I\big)\,, \tag{4}$$

$$z_t = u_t + \epsilon_t. \tag{5}$$

We refer to ϵ_t as the **learnable stochastic guidance**. The mean $\mu_\theta(u_t)$ encodes a state-dependent direction in which the trajectory is steered, while the variance $\sigma^2_\theta(u_t)$ controls the amount of exploration. This design allows GRAM to capture uncertainty, prevent convergence to local minima, and support robust exploration of the solution space without discarding the deterministic refinement of u_t.

Hierarchical Instantiation. We instantiate the latent state with two interacting components, $z = (h, l)$. The high-level component h is updated once per latent transition and carries abstract reasoning state, while the low-level component l is updated K times within a single transition and carries fine-grained intermediate computation. This decomposition separates the two roles across time scales, with h accumulating slowly across transitions and l refined rapidly within each one.

With this hierarchical multi-scale structure, a single transition $z_{t-1} \to z_t$ is computed as follows. The low-level component is first refined for K updates, with the high-level component held fixed:

$$l_{t,k} = f_{\mathrm{L}}(h_{t-1},\, l_{t,k-1},\, e_x;\, \theta), \qquad k = 1, \ldots, K, \tag{6}$$

where $l_{t,0} := l_{t-1}$ and we write $l_t := l_{t,K}$ for the refined low-level component. The high-level component is then updated as a stochastic transition conditioned on the refined l_t,

$$u_t = f_{\mathrm{H}}(h_{t-1},\, l_t;\, \theta), \tag{7}$$

$$\epsilon_t \sim p_\theta(\epsilon_t \mid u_t) := \mathcal{N}\big(\mu_\theta(u_t),\, \sigma^2_\theta(u_t)\, I\big), \tag{8}$$

$$h_t = u_t + \epsilon_t, \tag{9}$$

and we set $z_t = (h_t, l_t)$. Note that stochasticity is introduced only at the high level: the low-level refinement is fully deterministic, while the stochastic guidance signal ϵ_t acts on the slower, more abstract component of the latent state, where it can steer the overall reasoning trajectory across transitions. Under this instantiation, the decoder reads only the high-level component, i.e., $f_{\mathrm{dec}}(z_T) = f_{\mathrm{dec}}(h_T)$. Architectural details are in Appendix B.1, and Appendix D.8 ablates the level at which stochasticity is injected.

Modeling Unconditional Distribution. While the description so far focuses on the conditional setting $p_\theta(y \mid x)$, the same recursive process can also be defined as an unconditional generative model $p_\theta(x)$ when the input is replaced with an empty conditioning embedding. We use this formulation for unconditional generation tasks in Section 4.3.

2.2 Training

GRAM is trained to model the conditional distribution $p_\theta(y \mid x)$, where each training example consists of an input x and its corresponding target y. As a probabilistic model, GRAM adopts a latent-variable formulation and is optimized by maximizing an evidence lower bound (ELBO) with respect to the generative parameters θ and variational parameters ϕ.

Latent Variable Modeling. We model GRAM as a latent-variable probabilistic model p_θ, where the full latent trajectory $\tau = (z_0 \to \cdots \to z_{T_{\mathrm{Total}}})$ consists of a sequence of latent variables, with $T_{\mathrm{Total}} = T \times N_{\mathrm{sup}}$. The conditional likelihood is defined as

$$p_\theta(y \mid x) = \int p_\theta(y \mid \tau, x)\, p_\theta(\tau \mid x)\, d\tau\,, \tag{10}$$

where x denotes the input problem and y denotes the corresponding ground-truth output.

Direct maximum likelihood estimation of $\log p_\theta(y \mid x)$ is intractable due to the marginalization over latent trajectories. We therefore introduce a variational posterior $q_\phi(\tau \mid x, y)$ and optimize the evidence lower bound (ELBO), jointly training θ and ϕ via variational inference:

$$\log p_\theta(y \mid x) \geq \mathbb{E}_{q_\phi(\tau \mid x, y)}[\log p_\theta(y \mid \tau, x)] - \mathrm{KL}(q_\phi(\tau \mid x, y) \,\|\, p_\theta(\tau \mid x))\,. \tag{11}$$

During training, latent trajectories are sampled from the variational posterior $q_\phi(\cdot \mid x, y)$, which has access to both the input problem x and the target output y. At inference time, where y is unavailable, trajectories are instead generated from the learned prior $p_\theta(\cdot \mid x)$.

Both the prior and the posterior are modeled as conditional Markov processes over latent states:

$$p_\theta(\tau \mid x) = p(z_0) \prod_{t=1}^{T_{\mathrm{Total}}} p_\theta(z_t \mid z_{t-1}, x)\,, \qquad q_\phi(\tau \mid x, y) = p(z_0) \prod_{t=1}^{T_{\mathrm{Total}}} q_\phi(z_t \mid z_{t-1}, x, y)\,. \tag{12}$$

Here, z_0 is a fixed initial state shared by the prior and posterior. Both transitions are implemented by adding reparameterized Gaussian noise ϵ_t after a deterministic update u_t. The prior and the posterior differ only in the noise head applied on top of u_t, and the two heads have identical architecture but separate weights. The prior head parameterizes $p_\theta(\epsilon_t \mid u_t)$ as in Equation (8); the posterior head parameterizes $q_\phi(\epsilon_t \mid u_t, y) = \mathcal{N}\big(\mu_\phi(u_t, y),\, \sigma_\phi^2(u_t, y) I\big)$. Implementation details of both heads are given in Appendix B.1.

Since the two processes share the same Markov structure and all stochasticity is introduced through $\epsilon_{1:T_{\text{Total}}}$, their trajectory distributions can be equivalently represented in noise space. Moreover, since GRAM decodes the output only from the terminal latent state, the likelihood term satisfies $p_\theta(y \mid \tau, x) = p_\theta(y \mid z_{T_{\text{Total}}}, x)$. Therefore, the full trajectory-level ELBO can be written as

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi}\big[\log p_\theta(y \mid z_{T_{\text{Total}}}, x)\big] - \sum_{t=1}^{T_{\text{Total}}} \mathbb{E}_{q_\phi(\epsilon_{<t}\mid x,y)}\Big[\mathrm{KL}(q_\phi(\epsilon_t \mid u_t, y) \,\|\, p_\theta(\epsilon_t \mid u_t))\Big]. \quad (13)$$

Here, $u_t = f_{\text{H}}(h_{t-1}, l_t)$ denotes the deterministic high-level update before noise injection, as defined in Equation (7). Since u_t depends on h_{t-1}, which is determined by the previously sampled noise variables $\epsilon_{<t} := (\epsilon_1, \ldots, \epsilon_{t-1})$, the expectation averages over these ancestral samples.

Practical Implementation. In practice, following previous recursive reasoning models [8, 9], we train GRAM with deep supervision over N_{sup} consecutive supervision steps, each consisting of T recursive latent transitions. This provides dense learning signals along the full latent trajectory, rather than supervising only the final state after $T_{\text{Total}} = T \times N_{\text{sup}}$ transitions. The terminal state of each step is reused as the initial state of the next step.

Following standard practice for recurrent models with long computation chains, we apply truncated gradient propagation [16, 17], as used in recent recursive reasoning models [8, 9, 18]. In our implementation, gradients are propagated only through the final transition of each supervision step, $z_{T-1}^{(n)} \to z_T^{(n)}$. This gives the following surrogate objective for each supervision step:

$$\mathcal{L}_{\text{GRAM}}^{(n)}(x, y; \theta, \phi) = \mathbb{E}_{q_\phi}\big[\log p_\theta(y \mid z_T^{(n)}, x)\big] - \mathrm{KL}\big(q_\phi(\epsilon_T^{(n)} \mid u_T^{(n)}, y) \,\|\, p_\theta(\epsilon_T^{(n)} \mid u_T^{(n)})\big)\,, \quad (14)$$

where $z_T^{(n)}$ is the terminal state of the current supervision step n, and gradients are stopped through preceding states. Thus, $\mathcal{L}_{\text{GRAM}}$ should be viewed as a truncated surrogate objective, a biased but memory-efficient approximation to the exact ELBO. Further analysis of this approximation and the training hyperparameters are provided in Appendix A.2 and B.2, respectively. We additionally train an auxiliary halting head alongside the main objective, described in Appendix A.3.

2.3 Inference-Time Scaling

GRAM supports two complementary axes of inference-time scaling: *depth*, by varying the number of recursive transitions, and *width*, by sampling multiple latent reasoning trajectories in parallel. For depth, we follow prior recursive reasoning models [8, 9], where increasing the number of transitions at inference continues to improve performance, as shown in Figure 4. For width, which the stochastic transitions make available, we draw $\{\tau^{(i)}\}_{i=1}^{N} \sim p_\theta(\tau \mid x)$ from the learned prior and decode each terminal state into a candidate output $\hat{y}^{(i)} = f_{\text{dec}}(z_T^{(i)})$, exploring multiple stochastic reasoning paths simultaneously rather than extending a single trajectory.

To select among candidates, we use majority voting [19], which returns the most frequent prediction and adds no parameters or training, so the reported gains are attributable to parallel sampling itself. This improves robustness and solution quality without increasing the sequential recursion length, and the complete procedure is summarized in Algorithm 2. Other selectors are possible, such as the Latent Process Reward Model (LPRM) described in Appendix A.4.

3 Related Work

Latent Reasoning. Latent reasoning aims to reduce the inefficiency and verbosity of explicit Chain-of-Thought (CoT) by shifting part or all of the reasoning process into latent or continuous representations [1–6]. By avoiding token-by-token generation of intermediate steps, such representations can make reasoning traces more compact and reduce generation overhead. Existing approaches

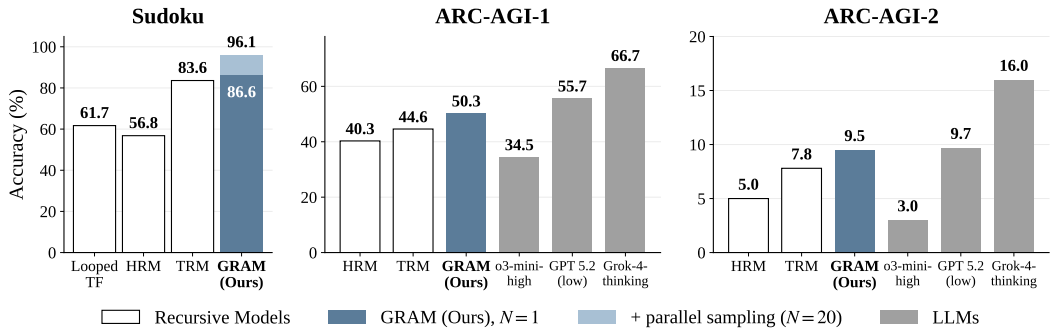


Figure 3: Performance on puzzle benchmarks. GRAM outperforms all deterministic recursive baselines on both Sudoku-Extreme and ARC-AGI even without parallel sampling. On Sudoku-Extreme, the GRAM bar is decomposed into its single-sample accuracy (N=1) and the further gain from parallel sampling (N=20). The ARC-AGI bars are reported without parallel sampling, since the standard pass@2 protocol (Appendix C.1) already aggregates predictions over 1k augmented inputs. Without test-time augmentation, parallel sampling still improves GRAM on ARC-AGI (Appendix D.3). Large reasoning model scores are external reference points for benchmark difficulty, and full results are in Table 8.

instantiate this idea through hidden states, latent or soft tokens, continuous thoughts, internal reasoning traces, and recursive state updates for scaling test-time computation [4, 7, 20–24, 18, 25–27]. However, many remain organized around autoregressive sequence generation, where additional computation is tied to generating more tokens, latent positions, or sequential reasoning states.

Recursive Architectures. Recursive architectures perform iterative state updates and have evolved from RNNs to weight-sharing Transformers with adaptive computation [7, 10, 28–33, 26]. Recent recursive reasoning models show that increasing inference-time depth can outperform larger static models [8, 9, 18, 25]. GRAM builds on this line but formulates recurrence as a probabilistic process: instead of following a single deterministic refinement path, it maintains stochastic latent trajectories, enabling multi-path exploration and generative sampling.

Probabilistic Latent State-Space Models. Probabilistic recurrent models use stochastic latent transitions to capture uncertainty and multimodal dynamics, often trained with variational inference [34–39]. They have been widely used in sequential generative modeling, video prediction, and model-based reinforcement learning. GRAM shares this latent state-space view but reinterprets stochastic dynamics as computation rather than temporal observation modeling: latent transitions define possible reasoning trajectories, supporting multi-hypothesis exploration and both conditional $p_\theta(y \mid x)$ and unconditional $p_\theta(x)$ generation.

4 Experiments

We evaluate GRAM against deterministic recursive baselines on tasks designed to probe what stochastic recursion changes. Stochastic transitions can benefit reasoning in two ways: during training, they expose the model to multiple trajectories for the same input, acting as a form of data augmentation; at inference, they allow the model to explore multiple trajectories and select among them. We therefore report both single-sample (N=1) and multi-sample (N=20) results. The N=1 setting isolates the training-time benefit by comparing all models on equal terms, while N=20 additionally captures the benefit of inference-time search, which deterministic models cannot exploit.

4.1 Challenging Puzzle Tasks

Setup. We evaluate on Sudoku-Extreme [8], which contains 9×9 puzzles with minimal clues requiring extensive constraint propagation, and ARC-AGI Challenge [13, 14], which tests abstract visual reasoning through few-shot pattern recognition. For Sudoku-Extreme, we report GRAM with a single sample and with N=20 parallel samples aggregated by majority voting. For ARC-AGI, we report results without parallel sampling, since the standard pass@2 protocol (Appendix C.1) already aggregates predictions over 1,000 augmented inputs. We compare against direct prediction (Transformer) and recursive baselines (Looped TF, HRM, TRM). Looped TF is evaluated on Sudoku-Extreme only, as training it on ARC-AGI is prohibitively expensive. On Sudoku-Extreme, the

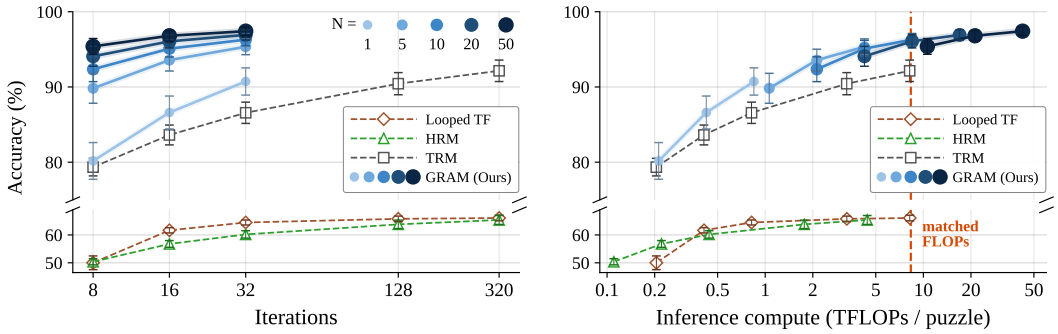


Figure 4: Inference-time scaling on Sudoku-Extreme. (Left) Recursion depth and parallel sampling. While all models benefit from longer recursion (x-axis), GRAM additionally scales with parallel sampling (N = number of samples). Each iteration corresponds to a supervision step, which amounts to $K\times$ more flat iterations for Looped TF. **(Right) Accuracy versus inference compute.** In FLOPs analysis, GRAM is more accurate than all baselines at every budget. The dashed line marks equal compute for TRM at 320 iters and GRAM at 16 iters with N=20. Curves show the mean over 3 seeds with one standard deviation.

published results of TRM [9] (87.4%) and HRM [8] (55.0%) are single runs, which neither capture variance across seeds nor are directly comparable to our 3-seed means. We therefore reproduce all recursive baselines, including those in the scaling analysis, with their official code and configurations and report the mean over 3 seeds. Large reasoning model results [40] are included as external reference points for benchmark difficulty, not as controlled baselines.

Probabilistic Recursion Improves Structured Reasoning. With a single sample, GRAM outperforms all recursive baselines on both benchmarks (Figure 3), reaching 86.6% against TRM's 83.6% on Sudoku-Extreme, 50.3% against 44.6% on ARC-AGI-1, and 9.5% against 7.8% on ARC-AGI-2. Parallel sampling lifts GRAM further to 96.1% on Sudoku-Extreme. We attribute this single-sample lead to training, where stochastic transitions expose GRAM to diverse intermediate states rather than a single fixed path. Full results, including large reasoning models, are given in Table 8 (Appendix D.1).

Parallel Sampling Provides a New Inference-Time Scaling Axis. Parallel sampling adds a scaling axis that deterministic models lack. As Figure 4 (left) shows, more samples improve GRAM on Sudoku-Extreme at every iteration count, and at 16 iterations accuracy rises from 86.6% at N=1 to 93.6% at N=5 and 96.1% at N=20. Drawing 20 samples multiplies the compute accordingly, yet GRAM uses this compute more effectively than deeper recursion, reaching 96.1% with 8.48 TFLOPs per puzzle, whereas TRM given nearly the same compute through 320 iterations reaches 92.1% with 8.19 TFLOPs (Figure 4, right). Moreover, because the samples run in parallel, this compute takes far less time, 184 ms per puzzle against 2,755 ms for TRM, whose recursion steps must run sequentially. Appendix D.2 reports the compute and latency measurements in detail.

4.2 Multi-solution Puzzle Tasks

Setup. To evaluate whether GRAM can solve problems whose inputs have multiple valid solutions, we test on N-Queens (8×8, 10×10) and Graph Coloring (8-vertex, 10-vertex). We compare against direct prediction (Transformer), recursive models (Looped TF, HRM, TRM), and generative models (MDLM [41] and an autoregressive Transformer, denoted AR). We report accuracy, whether the output satisfies all constraints, for N-Queens, and conflict edges, the number of constraint violations, for Graph Coloring. Both are reported in Table 1 from a single output of every model, and also with 20 samples for AR, MDLM, and GRAM. Coverage, the fraction of valid solutions recovered, is measured with 20 samples. Appendix C.2 describes how both datasets are constructed.

Deterministic Recursion Fails on Multi-Solution Tasks. Table 1 shows that deterministic recursive models fall behind in both accuracy and coverage on every multi-solution task. On 8×8 N-Queens, they reach at most 78.7% accuracy against 99.5% for GRAM and recover at most 26.7% of valid solutions, whereas sampling models recover 84.8% or more. The gap widens as inputs admit more solutions: from 8×8 to 10×10, where the average number of solutions per input rises from 1.65 to 2.93, the accuracy gap between TRM and GRAM grows from 32.7 to 72.2 points and deterministic coverage drops to at most 6.2%. Graph Coloring shows the same trend from 8 to 10 vertices, and grouping N-Queens inputs by solution count confirms it at the instance level: deterministic models

Table 1: Evaluation on N-Queens and Graph Coloring benchmarks. Rec. and Gen. indicate whether the model uses recursive computation and generative sampling, respectively. Values are mean ± standard deviation over runs, with accuracy and coverage in percent. N is the number of samples for accuracy and conflicts.

					N-Queens				Graph Coloring			
					8×8		10×10		8-vertex		10-vertex	
Method	Rec.	Gen.	# Params	N	Acc.	Cov@20	Acc.	Cov@20	Conflict↓	Cov@20	Conflict↓	Cov@20
Direct Pred (8 layers)	✗	✗	27M	–	$40.4_{\pm 1.1}$	$13.7_{\pm 1.1}$	$13.6_{\pm 0.5}$	$1.6_{\pm 0.2}$	$179.3_{\pm 4.0}$	$19.9_{\pm 0.2}$	$198.7_{\pm 5.0}$	$6.7_{\pm 0.1}$
Direct Pred (32 layers)	✗	✗	100M	–	$40.2_{\pm 1.3}$	$13.6_{\pm 1.1}$	$13.1_{\pm 0.4}$	$1.6_{\pm 0.2}$	$174.0_{\pm 18.0}$	$19.1_{\pm 1.7}$	$227.7_{\pm 34.5}$	$6.5_{\pm 1.9}$
Looped TF	✓	✗	7M	–	$68.1_{\pm 7.4}$	$23.6_{\pm 1.9}$	$50.0_{\pm 7.6}$	$6.2_{\pm 3.2}$	$136.0_{\pm 16.1}$	$20.5_{\pm 1.5}$	$157.3_{\pm 9.0}$	$7.2_{\pm 0.7}$
HRM	✓	✗	27M	–	$78.7_{\pm 2.9}$	$26.7_{\pm 1.3}$	$37.8_{\pm 0.7}$	$4.7_{\pm 0.1}$	$109.7_{\pm 1.5}$	$21.8_{\pm 0.3}$	$164.3_{\pm 21.6}$	$8.9_{\pm 1.7}$
TRM	✓	✗	7M	–	$66.8_{\pm 5.6}$	$22.6_{\pm 0.8}$	$17.5_{\pm 11.2}$	$2.0_{\pm 1.3}$	$109.3_{\pm 3.1}$	$22.3_{\pm 0.6}$	$170.7_{\pm 17.9}$	$6.8_{\pm 0.3}$
AR	✗	✓	10.6M	1	$96.3_{\pm 1.0}$	–	**90.0**$_{\pm 2.2}$	–	$37.3_{\pm 6.1}$	–	$101.7_{\pm 4.0}$	–
				20	$98.3_{\pm 0.7}$	$84.8_{\pm 0.8}$	$95.1_{\pm 1.5}$	$53.2_{\pm 0.8}$	$19.0_{\pm 11.3}$	$83.0_{\pm 0.7}$	$61.3_{\pm 8.3}$	$40.0_{\pm 0.3}$
MDLM	✗	✓	12.6M	1	$96.1_{\pm 1.5}$	–	$74.3_{\pm 6.6}$	–	$16.3_{\pm 2.5}$	–	$57.3_{\pm 4.7}$	–
				20	**100.0**$_{\pm 0.0}$	$87.2_{\pm 0.6}$	$98.6_{\pm 0.8}$	$47.4_{\pm 2.2}$	**2.7**$_{\pm 0.6}$	$84.5_{\pm 4.0}$	$12.0_{\pm 7.0}$	$48.2_{\pm 1.4}$
GRAM (Ours)	✓	✓	10M	1	**99.5**$_{\pm 0.3}$	–	$89.7_{\pm 2.7}$	–	**8.3**$_{\pm 2.5}$	–	**31.3**$_{\pm 15.0}$	–
				20	**100.0**$_{\pm 0.0}$	**89.7**$_{\pm 1.7}$	**100.0**$_{\pm 0.0}$	**57.5**$_{\pm 3.4}$	**2.7**$_{\pm 2.1}$	**85.8**$_{\pm 0.5}$	**3.3**$_{\pm 1.5}$	**51.3**$_{\pm 2.8}$

lose accuracy as solutions increase, while GRAM stays flat even with a single sample (Appendix D.4). Inputs with more valid solutions carry more conflicting targets, which a deterministic model must collapse into a single output, whereas GRAM absorbs this ambiguity in its latent trajectory.

Recursive Refinement Yields Sharper Constraint Satisfaction. Generative models (AR, MDLM) already achieve high coverage, but GRAM satisfies constraints more reliably in most settings while attaining the highest coverage in all four. It reaches 99.5% accuracy on 8×8 N-Queens and leaves under a third of AR's conflict edges on Graph Coloring. Even when all generative models draw 20 samples, GRAM still exceeds them on most tasks (Table 1). Together, these results suggest that recursive refinement enables stricter constraint satisfaction than generative sampling alone.

4.3 Exploring GRAM as an Unconditional Generator

Setup. To test GRAM's unconditional generative capability, we evaluate it on Sudoku generation from empty boards, measured by the fraction of valid boards, and on binarized MNIST generation, measured by IS [42] and FID [43]. GRAM replaces the input with an empty conditioning signal and samples from its learned prior. We compare with D3PM [44] on both tasks, and additionally with VAE [45] and TRM on MNIST. All models have roughly matched parameter counts; on Sudoku generation task, we also evaluate a D3PM about five times larger to test whether its performance is capacity-limited. Further details are in Appendix B.2.

Probabilistic Recursion Generalizes to Unconditional Generation. Unconditional generation tests the formulation directly. With no input, the entire output must come from the latent trajectory. Deterministic recursion therefore follows the same trajectory for every sample and collapses to a single output, with TRM producing a near-constant output on binarized MNIST. GRAM instead samples diverse digits from its prior, outperforming VAE with 16 steps and surpassing D3PM on both IS and FID with 256 steps, still far fewer than D3PM's 1,000 steps (Table 2). The gap widens on structured generation, where GRAM produces 99.05% valid Sudoku boards, all distinct among 100K samples, whereas D3PM reaches at most 79.18% even at five times GRAM's size (Figure 5). Figure 7 contrasts the generation trajectories

Table 2: Unconditional generation results on binarized MNIST. We report IS (↑) and FID (↓). For iterative models, a step corresponds to a supervision step for TRM and GRAM, and a denoising step for D3PM. FID is calculated using real samples with original pixel values (0–255).

Method	IS (↑)	FID (↓)
VAE	1.70	86.28
D3PM (1000 steps)	1.86	74.03
TRM (16 steps)	1.00	303.29
GRAM (Ours)		
16 steps	1.89	77.79
64 steps	1.95	75.39
256 steps	**2.04**	**73.34**

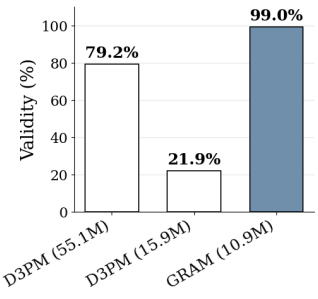
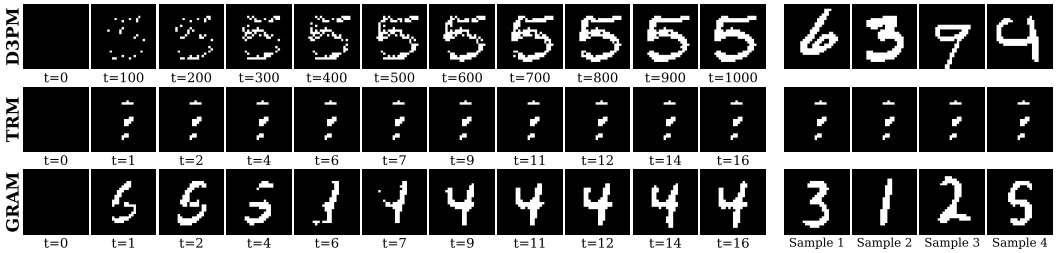


Figure 5: Unconditional Sudoku generation. Validity (%) of generated Sudoku puzzles. GRAM achieves higher validity than D3PM with substantially fewer parameters and steps.



(a) Generation Process (b) Samples

Figure 6: Visualization of the generation process and samples. (a) The generation process over recursion steps. Each row corresponds to a different model. GRAM (bottom) progressively refines the generated image through recursive latent updates, correcting initial errors. (b) Unconditional generated samples from each model.

Table 3: Ablation study on Sudoku-Extreme and N-Queens (8×8). Means and sample standard deviations over three seeds. (a) compares architectural variants built from the Looped TF baseline (DS = deep supervision, HR = hierarchical recursion, SG = stochastic guidance). (b) ablates the stochasticity and guidance mechanisms.

(a) Architecture Ablation.

	Sudoku		N-Queens	
Model variant	N=1	N=20	N=1	N=20
Base (Looped TF)	$61.7_{\pm 0.9}$	$61.7_{\pm 0.9}$	$68.1_{\pm 7.4}$	$68.1_{\pm 7.4}$
+ DS + HR (=TRM)	$83.6_{\pm 1.3}$	$83.6_{\pm 1.3}$	$66.8_{\pm 5.6}$	$66.8_{\pm 5.6}$
+ SG	$65.1_{\pm 0.5}$	$74.0_{\pm 0.6}$	$87.0_{\pm 7.5}$	$87.3_{\pm 7.8}$
+ DS + SG	$76.9_{\pm 2.6}$	$78.8_{\pm 1.5}$	$82.6_{\pm 10.7}$	$83.2_{\pm 10.2}$
+ DS + HR + SG (=GRAM)	$86.6_{\pm 2.2}$	**96.1**$_{\pm 0.9}$	$99.5_{\pm 0.3}$	**100.0**$_{\pm 0.0}$

(b) Mechanism Ablation (N=20).

Model variant	**Sudoku**	**N-Queens**
GRAM (ours)	**96.1**$_{\pm 0.9}$	**100.0**$_{\pm 0.0}$
w/o stochastic guidance	$83.6_{\pm 1.3}$	$66.8_{\pm 5.6}$
stochasticity only	$96.0_{\pm 1.1}$	$62.1_{\pm 14.9}$
guide only	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$
w/ direct prediction	$70.7_{\pm 7.6}$	$98.8_{\pm 2.2}$
TRM w/ stochastic decoder	$79.7_{\pm 1.9}$	$65.9_{\pm 4.7}$
TRM w/ random init	$91.2_{\pm 0.9}$	$71.4_{\pm 8.5}$

of GRAM and D3PM, suggesting that refining a complete candidate suits hard-constraint generation better than filling cells progressively.

Inference-Time Scaling Transfers to Generation Tasks. Table 2 shows that increasing recursive inference steps from 16 to 256 monotonically improves both IS ($1.89 \rightarrow 2.04$) and FID ($77.79 \rightarrow 73.34$), despite training with only 16 supervision steps. This shows that GRAM can use additional recursive computation at inference to improve generation quality. Figure 6 visualizes the recursive refinement process; additional samples from GRAM and the VAE baseline are provided in Appendix D.5.

4.4 Ablation Study

We ablate GRAM's model structure and stochastic mechanism on Sudoku-Extreme and N-Queens (8×8). Table 3 reports the architecture ablation in (a) and the stochasticity ablation in (b).

Stochastic Guidance Provides Consistent Gains Across Architectures. Table 3a shows that stochastic guidance (SG) improves both the flat Looped TF baseline and the hierarchically recursive TRM variant. With SG, Looped TF reaches 74.0% on Sudoku and 87.3% on N-Queens at $N = 20$. Combined with deep supervision and hierarchical recursion, SG yields the full GRAM model, achieving the best overall performance (96.1%). These results demonstrate that SG remains effective across different model structures.

Stochasticity and Guidance Play Complementary Roles. Table 3b ablates each component of the learned distribution $\epsilon_t \sim \mathcal{N}(\mu_\theta, \sigma_\theta^2 I)$ in Equation (4). We report N=20, where the effect of stochasticity is most visible; single-sample results in Appendix D.7 lead to the same conclusions. Removing guidance (*stochasticity only*, $\epsilon_t \sim \mathcal{N}(0, \sigma_\theta^2 I)$) retains strong performance on Sudoku (96.0%) but degrades substantially on N-Queens (62.1%). Removing stochasticity (*guide only*, $\epsilon_t = \mu_\theta$) fails completely on both tasks (0.0%). Stochasticity thus provides the exploration that recursion relies on, while guidance steers this exploration toward valid solutions, which matters most when multiple solutions compete as in N-Queens. Beyond these two components, the *w/ direct prediction* variant samples $z_t \sim \mathcal{N}(\mu_\theta(u_t), \sigma_\theta^2(u_t) I)$ directly instead of using the residual update $z_t = u_t + \epsilon_t$, and thus no longer recovers the deterministic update $z_t = u_t$ when $\epsilon_t = 0$. Its

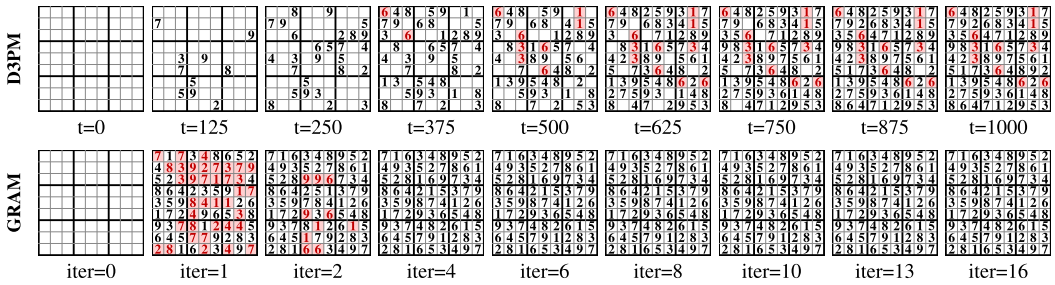


Figure 7: Generation trajectories of D3PM and GRAM on unconditional Sudoku. Each row shows successive states of a single generation from an empty grid, D3PM over denoising steps t and GRAM over recursion steps. GRAM proposes a complete board at its first step and corrects it in place, while D3PM fills cells progressively. Incorrect digits are highlighted in red.

inconsistent performance across Sudoku (70.7%) and N-Queens (98.8%) suggests that separating deterministic recursion from stochastic guidance yields a more robust transition structure.

Variational Stochastic Recursion Outperforms Simple Randomization. We test two simple approaches to add stochasticity to TRM: (1) *stochastic decoding*, which samples from the output distribution instead of argmax, and (2) *random initialization*, which samples z_0 from a Gaussian $\mathcal{N}(0, I)$ at each inference. Stochastic decoding provides no meaningful improvement over deterministic TRM. With majority voting at $N = 20$, random initialization improves Sudoku to 91.2%, but remains below GRAM (96.1%). On N-Queens, it provides only a modest improvement over TRM (66.8% to 71.4%), remaining far below GRAM (100.0%). These results suggest that variationally modeling stochastic transitions within the recursive process is important for achieving consistent gains.

5 Conclusions and Limitations

We introduced GRAM, a generative framework that transforms deterministic recursive architectures into probabilistic generative models capable of modeling both $p(y \mid x)$ and $p(x)$ via recursive amortized variational inference. For reasoning problems, introducing stochasticity into latent transitions enables diverse solution discovery and improved exploration compared to deterministic counterparts. Notably, we demonstrate GRAM can leverage width-based inference-time scaling as a complement to depth, by sampling multiple latent trajectories in parallel and bypassing the latency bottleneck of depth-only scaling. Our ablations further reveal that stochastic guidance is a general-purpose extension that improves recursion whether flat or hierarchical, and that the gains stem specifically from the variational framework — not from mere randomness, as naive stochastic alternatives applied to existing models fall well short of GRAM. Beyond solution-seeking, GRAM also demonstrates potential as an unconditional generative model through recursion-based generation over inputs, with generation quality improving monotonically with recursive depth even beyond training-time steps.

These results suggest new directions for generative modeling via hierarchical recursion. Despite these strengths, the sequential nature of deep supervision limits training efficiency compared to Transformers, posing a significant barrier to scaling GRAM toward larger foundation models.

Acknowledgment

This research was supported by the GRDC (Global Research Development Center) Cooperative Hub Program (RS-2024-00436165) through the National Research Foundation of Korea (NRF) funded by the Ministry of Science and ICT (MSIT) and by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grants funded by the Korea government (MSIT) (No. RS-2024-00509279, Global AI Frontier Lab, NYU-KAIST Global Innovation and Research Institute, and No. RS-2026-25526850, HERMES-Net). We also acknowledge support from the Youlchon Foundation. Minsu Kim acknowledges funding from the KAIST Jang Young Sil Fellow Program. We are especially grateful to Gyubin and Seungju for their non-trivial contributions, and we thank the members of the MLML for valuable discussions and feedback throughout this project.

Broader Impacts

GRAM studies probabilistic recursive reasoning for structured reasoning and generation. By maintaining multiple latent trajectories, it may benefit tasks such as constraint satisfaction, and scientific problem solving, where uncertainty and multiple valid solutions are common. It also suggests a way to improve reasoning through inference-time computation rather than parameter scaling alone. Its generality also entails risks: plausible but invalid generations may be mistaken for verified solutions in downstream decision-making pipelines, and multi-sample inference may increase computational and energy costs at scale. Since our experiments focus on controlled benchmarks, deployment in real-world or high-stakes settings would require rigorous validation, uncertainty calibration, and domain-specific safeguards.

References

[1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[2] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.

[3] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 17682–17690, 2024.

[4] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.

[5] Hanlin Zhu, Shibo Hao, Zhiting Hu, Jiantao Jiao, Stuart Russell, and Yuandong Tian. Reasoning by superposition: A theoretical perspective on chain of continuous thought. *arXiv preprint arXiv:2505.12514*, 2025.

[6] Halil Alperen Gozeten, M Emrullah Ildiz, Xuechen Zhang, Hrayr Harutyunyan, Ankit Singh Rawat, and Samet Oymak. Continuous chain of thought enables parallel exploration and reasoning. *arXiv preprint arXiv:2505.23648*, 2025.

[7] Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms. *arXiv preprint arXiv:2311.12424*, 2023.

[8] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025.

[9] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025.

[10] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. *arXiv preprint arXiv:1807.03819*, 2018.

[11] Daniel Kahneman. Thinking, fast and slow. *Farrar, Straus and Giroux*, 2011.

[12] Yoshua Bengio. The consciousness prior. *arXiv preprint arXiv:1709.08568*, 2017.

[13] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[14] Francois Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. Arc-agi-2: A new challenge for frontier ai reasoning systems. *arXiv preprint arXiv:2505.11831*, 2025.

[15] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

[16] Ronald J Williams and Jing Peng. An efficient gradient-based algorithm for on-line training of recurrent network trajectories. *Neural computation*, 2(4):490–501, 1990.

[17] Corentin Tallec and Yann Ollivier. Unbiasing truncated backpropagation through time. *arXiv preprint arXiv:1705.08209*, 2017.

[18] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025.

[19] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

[20] Yufan Zhuang, Liyuan Liu, Chandan Singh, Jingbo Shang, and Jianfeng Gao. Text generation beyond discrete token sampling. *arXiv preprint arXiv:2505.14827*, 2025.

[21] Zhen Zhang, Xuehai He, Weixiang Yan, Ao Shen, Chenyang Zhao, Shuohang Wang, Yelong Shen, and Xin Eric Wang. Soft thinking: Unlocking the reasoning potential of llms in continuous concept space. *arXiv preprint arXiv:2505.15778*, 2025.

[22] Natasha Butt, Ariel Kwiatkowski, Ismail Labiad, Julia Kempe, and Yann Ollivier. Soft tokens, hard truths. *arXiv preprint arXiv:2509.19170*, 2025.

[23] Zhenyi Shen, Hanqi Yan, Linhai Zhang, Zhanghao Hu, Yali Du, and Yulan He. Codi: Compressing chain-of-thought into continuous space via self-distillation. *arXiv preprint arXiv:2502.21074*, 2025.

[24] Zhenrui Yue, Bowen Jin, Huimin Zeng, Honglei Zhuang, Zhen Qin, Jinsung Yoon, Lanyu Shang, Jiawei Han, and Dong Wang. Hybrid latent reasoning via reinforcement learning. *arXiv preprint arXiv:2505.18454*, 2025.

[25] Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, et al. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. *arXiv preprint arXiv:2507.10524*, 2025.

[26] Amirkeivan Mohtashami, Matteo Pagliardini, and Martin Jaggi. Cotformer: A chain-of-thought driven architecture with budget-adaptive computation cost at inference. *arXiv preprint arXiv:2310.10845*, 2023.

[27] Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed recursive transformers: Effective parameter sharing with layer-wise lora. *arXiv preprint arXiv:2410.20672*, 2024.

[28] Jeffrey L Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.

[29] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

[30] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

[31] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942*, 2019.

[32] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. *arXiv preprint arXiv:1910.10073*, 2019.

[33] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[34] Junyoung Chung, Kyle Kastner, Laurent Dinh, Kratarth Goel, Aaron Courville, and Yoshua Bengio. A recurrent latent variable model for sequential data, 2016. URL https://arxiv.org/abs/1506.02216.

[35] Marco Fraccaro, Søren Kaae Sønderby, Ulrich Paquet, and Ole Winther. Sequential neural models with stochastic layers, 2016. URL https://arxiv.org/abs/1605.07571.

[36] Rahul G. Krishnan, Uri Shalit, and David Sontag. Deep kalman filters, 2015. URL https://arxiv.org/abs/1511.05121.

[37] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels, 2019. URL https://arxiv.org/abs/1811.04551.

[38] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2020.

[39] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[40] ARC-Prize-Foundation. ARC-AGI benchmarking: Leaderboard and dataset for the ARC-AGI benchmark. https://arcprize.org/leaderboard, 2026. Accessed: 2026-1-22.

[41] Subham Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 37:130136–130184, 2024.

[42] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training gans. *Advances in neural information processing systems*, 29, 2016.

[43] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems*, 30, 2017.

[44] Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in neural information processing systems*, 34:17981–17993, 2021.

[45] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[46] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[47] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[48] Simo Ryu. Minimal implementation of a d3pm (structured denoising diffusion models in discrete state-spaces), in pytorch. https://github.com/cloneofsimo/d3pm, 2024.

[49] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023.

[50] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[51] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.

[52] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks*, 107:3–11, 2018.

[53] Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*, pages 3–19, 2018.

[54] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[55] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[56] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale gan training for high fidelity natural image synthesis. *arXiv preprint arXiv:1809.11096*, 2018.

[57] Yang Song and Stefano Ermon. Improved techniques for training score-based generative models. *Advances in neural information processing systems*, 33:12438–12448, 2020.

[58] Jordan Bell and Brett Stevens. A survey of known results and research areas for n-queens. *Discrete mathematics*, 309(1):1–31, 2009.

[59] P. Erdős and A. Rényi. On the strength of connectedness of a random graph. *Acta Mathematica Academiae Scientiarum Hungarica*, 12(1):261–267, Mar 1964. ISSN 1588-2632. doi: 10.1007/BF02066689. URL https://doi.org/10.1007/BF02066689.

[60] Henrique Lemos, Marcelo Prates, Pedro Avelar, and Luis Lamb. Graph colouring meets deep learning: Effective graph neural network models for combinatorial problems. In *2019 IEEE 31st International Conference on Tools with Artificial Intelligence (ICTAI)*, pages 879–885. IEEE, 2019.

[61] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[62] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

[63] Jonathon Shlens. A tutorial on principal component analysis. *arXiv preprint arXiv:1404.1100*, 2014.

[64] Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Commun. ACM*, 18:509–517, 1975. URL https://api.semanticscholar.org/CorpusID:13091446.

A Additional Method Details

A.1 Algorithms

Algorithm 1 and Algorithm 2 summarize training and inference. Both invoke the same latent transition, differing only in the distribution from which the stochastic guidance ϵ is drawn: the target-conditioned posterior q_ϕ during training, and the prior p_θ at inference, where y is unavailable.

Algorithm 1 GRAM training

Require: input x, target y;
Require: supervision steps N_{sup}, transition steps T, refinement steps K; KL weight β; halt head q_ψ;

procedure TRANSITION($z,\ e_x,\ \pi$)
  // π: posterior q_ϕ in training, prior p_θ at inference
  $(h, l) \leftarrow z$
  for $k = 1$ **to** K **do** ▷ low-level refinement
    $l \leftarrow f_{\text{L}}(l + h + e_x)$
  $u \leftarrow f_{\text{H}}(h + l)$ ▷ deterministic proposal, Equation (7)
  $\epsilon \sim \pi(\cdot \mid u)$ ▷ stochastic guidance
  return $\big((u + \epsilon,\ l),\ u\big)$

// Main loop
$e_x \leftarrow f_{\text{enc}}(x)$; $\quad z \leftarrow z_0$
for $n = 1$ **to** N_{sup} **do**
  // T transition steps (gradient flow only at T-th step)
  for $t = 1$ **to** $T - 1$ **do**
    $(z, \cdot) \leftarrow \text{stopgrad}\big(\text{TRANSITION}(z, e_x,\ q_\phi(\cdot \mid \cdot,\ y))\big)$
  $(z, u_T) \leftarrow \text{TRANSITION}(z, e_x,\ q_\phi(\cdot \mid \cdot,\ y))$

  // Compute loss (prediction + KL + ACT) and update weights
  $(h, \cdot) \leftarrow z$; $\quad \hat{y} \leftarrow \arg\max f_{\text{dec}}(h)$; $\quad q \leftarrow q_\psi(h)$
  $\mathcal{L}^{(n)} \leftarrow \text{CE}(f_{\text{dec}}(h),\ y)\ +\ \beta\,\text{KL}\big(q_\phi(\cdot \mid u_T, y) \,\|\, p_\theta(\cdot \mid u_T)\big)$ ▷ only $t = T$ contributes
  $\mathcal{L}^{(n)} \leftarrow \mathcal{L}^{(n)} + \text{BCE}\big(q,\ \mathbf{1}[\hat{y} = y]\big)$ ▷ ACT halting, Appendix A.3
  $(\theta, \phi, \psi) \leftarrow \text{ADAMW}\big((\theta, \phi, \psi),\ \nabla\mathcal{L}^{(n)}\big)$

  // detaching and carrying states
  $z \leftarrow \text{stopgrad}(z)$ ▷ no gradient across supervision steps
  if $\sigma(q) > 0.5$ **then break** ▷ halt when the prediction is judged correct

Algorithm 2 GRAM inference with parallel trajectory sampling

Require: input x;
Require: trajectories N, supervision steps N_{sup}, transition steps T; TRANSITION as in Algorithm 1

// Width: N trajectories drawn from the prior, computed in parallel
$e_x \leftarrow f_{\text{enc}}(x)$
for $i = 1$ **to** N **do**
  $z \leftarrow z_0$
  // Depth: fixed number of steps (halting disabled for fair comparison)
  for $n = 1$ **to** N_{sup} **do**
    for $t = 1$ **to** T **do**
      $(z, \cdot) \leftarrow \text{TRANSITION}(z, e_x,\ p_\theta(\cdot \mid \cdot))$ ▷ prior: y is unavailable
  $(h, \cdot) \leftarrow z$; $\quad \hat{y}^{(i)} \leftarrow \arg\max f_{\text{dec}}(h)$

// Select final prediction among N candidates
return $\arg\max_y \sum_{i=1}^{N} \mathbf{1}[\hat{y}^{(i)} = y]$ ▷ majority voting

A.2 Empirical Validation of the Surrogate Objective

We further analyze the approximation introduced by the surrogate training objective $\mathcal{L}_{\text{GRAM}}$ used in Section 2.2, both qualitatively and empirically.

Truncation as a gradient approximation. We frame $\mathcal{L}_{\text{GRAM}}$ as a gradient approximation rather than a separate variational objective. The full trajectory-level ELBO (Equation (13)) involves a sum of KL terms across all T_{Total} transitions, and computing its exact gradient requires backpropagation through the entire trajectory. To enable training with constant memory, we propagate gradients only through the final transition of each supervision step. This is a standard practice in recurrent latent variable models with long computation chains: ELBOs over truncated sequences are used, for example, in VRNN [34] and SRNN [35], while truncated latent imagination is used in Dreamer-family world models [38, 39]. Trading a small gradient bias for training stability via local truncation is therefore well-precedented; what is specific to GRAM is applying this approximation at the level of recursive reasoning trajectories rather than temporal sequences.

Empirical validation. To verify that optimizing $\mathcal{L}_{\text{GRAM}}$ effectively drives improvement in the full variational bound, we compute both quantities on the validation set throughout training. The full ELBO $\mathcal{L}_{\text{ELBO}}$ is evaluated as in Equation (13), summing the reconstruction term and KL contributions across all T_{Total} transitions; the surrogate objective is evaluated as the average of $\mathcal{L}_{\text{GRAM}}^{(n)}$ over the N_{sup} supervision steps. Figure 8 reports the results on Sudoku-Extreme and N-Queens 8×8.

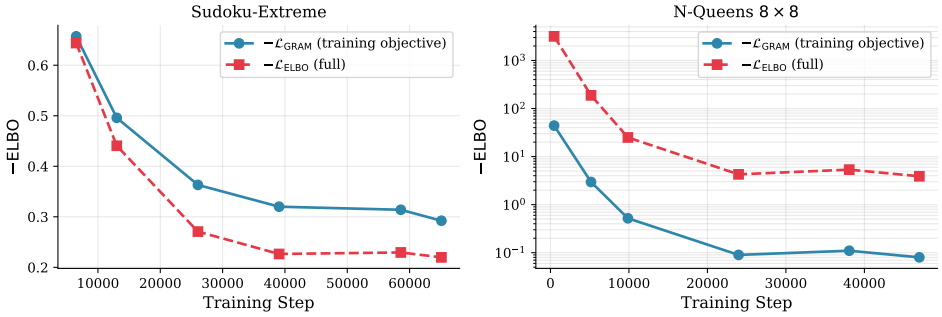


Figure 8: Full ELBO $\mathcal{L}_{\text{ELBO}}$ and surrogate objective $\mathcal{L}_{\text{GRAM}}$ throughout training (plotted as $-\text{ELBO}$, smaller is better). On both Sudoku-Extreme (left) and N-Queens 8×8 (right), both quantities decrease monotonically over training, indicating that gradient updates of $\mathcal{L}_{\text{GRAM}}$ consistently improve the full variational bound. The two curves do not coincide because $\mathcal{L}_{\text{ELBO}}$ sums KL contributions across all T_{Total} transitions while $\mathcal{L}_{\text{GRAM}}$ evaluates only the final-step KL of each supervision step; their gap reflects the cumulative KL across earlier transitions, not a failure of optimization. The N-Queens plot uses a log scale on the y-axis due to the large dynamic range.

Both $\mathcal{L}_{\text{ELBO}}$ and $\mathcal{L}_{\text{GRAM}}$ improve monotonically throughout training on both tasks. This indicates that, despite the truncation, gradient updates of $\mathcal{L}_{\text{GRAM}}$ effectively drive improvement in the full variational bound. Since $\mathcal{L}_{\text{ELBO}}$ also serves as an indirect estimate of the negative log-likelihood, its consistent improvement provides evidence that GRAM optimizes a well-defined data likelihood, even though training relies on the surrogate.

The gap between the two curves in Figure 8 reflects the structural difference between the two quantities — $\mathcal{L}_{\text{ELBO}}$ accumulates KL terms across all transitions while $\mathcal{L}_{\text{GRAM}}$ evaluates only the final-step KL of each supervision step — rather than an optimization failure. This gap is consistent with $\mathcal{L}_{\text{GRAM}}$ being a biased but useful surrogate for $\mathcal{L}_{\text{ELBO}}$.

A.3 Adaptive Computation Time

GRAM optionally adopts adaptive computation time (ACT) [8–10] at inference, allowing each trajectory to terminate at a learned halting depth rather than running for a fixed number of supervision steps. We follow the Q-learning formulation introduced by HRM [8] and adopted in TRM [9].

Halt head. The decoder includes an auxiliary head $q_\psi : \mathbb{R}^D \to \mathbb{R}^2$ that maps the high-level state h to two scalar values, $q_\psi(h) = (q^{\text{halt}}, q^{\text{continue}})$. These are interpreted as estimated Q-values for the binary action of halting or continuing computation at the current supervision step.

Training. The halt head is trained jointly with the main objective via a temporal-difference loss. After computing the latent state $z_T^{(n)}$ at the end of supervision step n, we form Q-learning targets:

- $\hat{q}_n^{\text{halt}} = \mathbf{1}[\hat{y}^{(n)} = y]$, indicating whether decoding the current state would yield a correct prediction.
- $\hat{q}_n^{\text{continue}} = \max\big(q_{n+1}^{\text{halt}},\, q_{n+1}^{\text{continue}}\big)$, the bootstrapped value of running one more supervision step.

The halt head is trained by regression to these targets:

$$\mathcal{L}_{\text{ACT}} = \sum_{n=1}^{N_{\text{sup}}} \Big[\big(q_n^{\text{halt}} - \hat{q}_n^{\text{halt}}\big)^2 + \big(q_n^{\text{continue}} - \hat{q}_n^{\text{continue}}\big)^2 \Big]. \tag{15}$$

This auxiliary loss is added to the main training objective.

Inference. At inference, computation proceeds one supervision step at a time. After each step n, we evaluate $q_\psi(h^{(n)})$ and halt if $q_n^{\text{halt}} > q_n^{\text{continue}}$, returning $\hat{y}^{(n)}$ as the prediction. Otherwise, computation continues to the next supervision step, up to a maximum budget of $N_{\text{sup}}^{\max}$ steps. Different trajectories sampled in parallel may therefore terminate at different depths, complementing the parallel-sampling scheme described in Section 2.3. In practice, we found that using only q^{halt} (halting when $\sigma(q^{\text{halt}}) > 0.5$, without the continue branch) performs comparably while simplifying implementation; our released code uses this variant.

A.4 Latent Process Reward Model (LPRM)

Parallel sampling produces N candidate outputs, and any rule that scores candidates can be used to select among them. We use majority voting throughout this paper. However, we also discuss a learned selector supported by our implementation, the Latent Process Reward Model (LPRM).

The LPRM is a value head $v_\psi(z_t)$ trained to predict the expected accuracy of the final output, conditioned on the current latent state z_t. It is trained jointly with the main objective via a regression loss

$$\mathcal{L}_{\text{LPRM}} = \sum_{t=1}^{T} \big(v_\psi(z_t) - r\big)^2, \tag{16}$$

where $r \in \{0, 1\}$ denotes the accuracy of the final prediction for a given trajectory. At inference, best-of-N selection returns $\hat{y}^{(n^*)}$ with $n^* = \arg\max_n v_\psi(z_T^{(n)})$, in place of the majority-voted prediction.

B Training and Architecture Details

B.1 Architecture Details

GRAM consists of three components: Encoder, Recursive Core, and Decoder.

Encoder. Input tokens are mapped to embeddings via a token embedding layer and combined with positional encodings (RoPE) [46]. The embeddings are scaled by $\sqrt{D}$ and prepended with 16 puzzle embedding tokens [8], produced from a learned embedding indexed by a puzzle identifier. On ARC-AGI [13, 14], each puzzle has its own identifier, so these tokens distinguish tasks. On all other tasks, a single identifier is shared across examples, and they instead act as CLS-like register tokens. These tokens are carried through the recursion, so the latent states h and l span these positions as well as the input tokens.

Recursive Core. The core maintains two latent states: h (high-level) and l (low-level). For each outer step, the low-level state is refined K times via $l \leftarrow f_L(l + h + e_x)$, injecting the input embedding at each iteration. The high-level state is then updated via $h \leftarrow f_H(h + l)$. f_L and f_H are implemented by a single network whose parameters are reused for both roles and across all latent transitions and supervision steps, so recursion depth can be increased without increasing the parameter count. In addition, as an exception, we use [SwiGLU + SwiGLU] network for the Recursive Core module

Table 4: Architecture components.

Component	Module	Description
Encoder		
	Token Embedding	vocab $\to D$
	Puzzle Embedding	16 tokens
	Position Encoding	RoPE
Recursive Core		
	f_L, f_H	[Attn. + SwiGLU] $\times$ 2 blocks
	Iterations	K low-level, T high-level steps
	Prior head $\mu_\theta, \sigma_\theta$	SwiGLU MLP, input u_t
	Posterior head μ_ϕ, σ_ϕ	SwiGLU MLP, input $[\, u_t \,;\, \mathrm{repr}(y) \,]$
Decoder		
	LM Head	SwiGLU MLP($D \to$ vocab)
	Q Head	Linear($D \to 2$) for halt

instead of [Attention + SwiGLU] for Sudoku tasks, following [9]. For initialization of $z_0 = (h_0, l_0)$, we sample once from the standard Gaussian distribution $\mathcal{N}(0, I)$, then save the value within the network checkpoint and load it again, meaning the initialized z_0 has a fixed value.

Stochastic Guidance Heads. The prior heads $(\mu_\theta, \sigma_\theta)$ and the posterior heads (μ_ϕ, σ_ϕ) are implemented as token-wise SwiGLU [47] MLPs, applied independently at each of the L sequence positions of the high-level state. The two have identical architecture but separate weights, and differ in their input: the prior head takes the deterministic update u_t, while the posterior head takes u_t concatenated along the feature dimension with a representation of the target y. For binarized-MNIST [15], this representation is produced by the convolutional patch encoder described in Table 5; for all other tasks, it is a one-hot encoding of y over the task vocabulary (Table 6).

Decoder. The decoder extracts content tokens from h (excluding puzzle embedding positions) and maps them to logits via a SwiGLU MLP head. An auxiliary head predicts halt decisions from the first puzzle embedding position of h.

Encoder and Decoder for Image Patches. In the MNIST [15] image generation task, we first construct a binarized dataset by normalizing the original discrete pixel values ($0 \sim 255$) to the continuous range $[0, 1]$ and applying a threshold at 0.5. For the network architecture, we employ a convolutional patch encoder, following [48, 49].

The encoding process proceeds in three stages. First, the discrete input tokens $x \in \{0, 1\}$ are normalized to the range $[-1, 1]$. Second, to capture local spatial dependencies before patchification, the normalized image passes through a shallow convolutional encoder. This encoder consists of two stacked blocks, where each block comprises a 2D convolution [50, 51] with a 5×5 kernel and padding 2, a SiLU non-linearity [52], and Group Normalization (GN) [53]. Finally, the resulting feature map is divided into non-overlapping patches of size $P \times P$ and linearly projected to match the model's hidden dimension D. The detailed architectural specifications and dimension transitions are summarized in Table 5.

Hyperparameters. Following [8, 9], both the input and output are represented as sequences of shape $[B, L]$, where B denotes the batch size and L the context length. Each input sequence includes 16 fixed puzzle embedding tokens. The latent states h_t and l_t, as well as the decoder output, have shape $[B, L, D]$, with embedding dimension D. The Transformer [54] backbone uses embedding dimension $D = 512$, attention heads N_{head}=8 , and FFN hidden dimension D_h=512. Each latent transition $z_t \to z_{t+1}$ applies K low-level (inner) steps followed by one high-level update, with $K = 6$ for Sudoku-Extreme [8] and $K = 4$ for all other tasks. Each supervision step consists of $T = 3$ such transitions.

B.2 Training Details

Task Configuration. All tasks represent inputs and outputs as discrete token sequences (Summarized in Table 6). Puzzle identifiers are task-specific only for ARC-AGI, and shared across examples otherwise.

Table 5: Detailed architecture of the Image Patch Encoder for binarized-MNIST. H, W denote image resolution, C input channels, P patch size, N_p the number of patches, and D the hidden dimension.

Stage	Layer / Operation	Output Dim.
1. Norm.	Input Tokens	(B, C, H, W)
	Linear Scaling $[-1, 1]$	(B, C, H, W)
2. Conv	Conv2d 5×5 $(p = 2)$ SiLU $\rightarrow$ GN(32)	$(B, D/2, H, W)$
	Conv2d 5×5 $(p = 2)$ SiLU $\rightarrow$ GN(32)	$(B, D/2, H, W)$
3. Patch	Flatten Patches	$(B, N_p, P^2 \cdot \frac{D}{2})$
	Linear Projection	(B, N_p, D)

- For *Sudoku-Extreme* [8], the 9×9 grid is flattened row-by-row into 81 tokens with vocabulary size 11 (0=pad, 1=blank, 2–10=digits). Unconditional Sudoku generation uses the same encoding, with an all-blank board as input and a complete solution as target.
- For *ARC-AGI* [13, 14], variable-size grids are padded to a fixed 30×30 canvas with EOS markers, yielding 900 tokens and vocabulary size 12 (0=pad, 1=eos, 2–11=colors); task-specific puzzle embeddings are prepended to distinguish different ARC tasks.
- *N-Queens* flattens an $N \times N$ board row-by-row into N^2 tokens with vocabulary size 3 (0=pad, 1=empty, 2=queen).
- *Graph Coloring* encodes the upper triangle of the adjacency matrix as $n(n+1)/2$ tokens, using 0=PAD, 1=no-edge, and 2=edge for inputs and $3 +$ color_id for output colors.
- For image generation on *binarized-MNIST* [15], images are quantized into pixels with vocabulary size 3 (0=pad, 1=black, 2=white) and processed via CNN-based patchification [50, 49], with the encoder applying patchify and the decoder unpatchify. Then, patched input forms 14×14 flattened sequence tokens.

Table 6: Task-specific configurations.

Dataset	Seq. Len	Vocab	Encoding
Sudoku	81	11	9×9 grid, row-major
ARC-AGI	900	12	30×30 padded canvas
N-Queens	N^2	3	$N \times N$ board
Graph Coloring	$\frac{n(n+1)}{2}$	6	Adjacency upper triangle
binarized-MNIST	196	3	14×14 patches

Training Details. We train GRAM using AdamW [55] with a learning rate of 10^{-4} and a weight decay of 1.0. The global batch size is 768. For stability, we apply exponential moving average (EMA) with decay 0.999, following Brock et al. [56] and Song and Ermon [57]. To prevent posterior collapse, we use a KL balance [38, 39] coefficient of 0.8. The number of deep supervision steps is $N_{\text{sup}} = 16$ for all tasks. The KL coefficient β is set to 0.1 (Sudoku), $0.04/0.1$ (ARC-AGI-1/2), $0.07/0.045$ (N-Queens $8 \times 8/10 \times 10$), $0.5/0.45$ (Graph Coloring with 8/10 nodes), and 0.015 (binarized-MNIST). Task-specific training configurations are summarized in Table 7.

binarized-MNIST Model and Training Configurations. For the comparison, we select VAE, TRM, D3PM, and GRAM configurations with comparable parameter counts. The VAE uses a 2-layer convolutional encoder with a base channel width of 128, separate MLP heads for the posterior mean and log-variance, a 384-dimensional latent space, and a transposed-convolutional decoder, resulting in 15.81M parameters. D3PM uses a convolutional patch encoder followed by a 3-layer DiT-style Transformer with hidden dimension 512 and 8 attention heads, where diffusion timesteps condition each block through adaptive layer normalization (adaLN) [49]. It uses an absorbing-state corruption process with 1,000 diffusion steps and contains 18.07M parameters. GRAM uses a 2-layer recursive Transformer core with hidden dimension 512, 8 attention heads, and a 256-dimensional stochastic

latent space, resulting in 15.82M parameters. TRM, the deterministic recursive baseline, has 10.90M parameters.

Because D3PM and GRAM treat pixels as categorical variables, we train the VAE with a binary cross-entropy reconstruction term to maintain a consistent binary-pixel representation across the three models.

Unconditional Sudoku Generation Model and Training Configurations. Unlike the conditional Sudoku-solving setting, where the input board contains given clues, the model receives an entirely blank board and samples a complete 9×9 Sudoku board from its learned prior. We evaluate each generated board using the standard Sudoku validity criterion: every row, column, and 3×3 box must contain the digits 1 through 9 exactly once. We report the validity rate over 100K generated boards. To check whether high validity comes from repeatedly producing the same board, we also compute the fraction of unique boards among valid samples.

For GRAM, we construct the unconditional training set from Sudoku-Extreme [8], the Sudoku benchmark used by HRM and TRM. We sample 50K complete solutions from the original training split, discard the clue patterns, and use an all-blank board as input with the complete solution as the target. No data augmentation is used. We train GRAM on this derived 50K-solution set for 200 epochs with learning rate 10^{-4}, EMA decay 0.999, and KL coefficient 0.05. The resulting model contains 10.9M parameters and uses 16 inference steps.

For D3PM, we use a DiT-style Transformer backbone with the absorbing-state noise process, and evaluate two model sizes. D3PM-Big uses hidden dimension 768, 5 Transformer blocks, and 12 attention heads, yielding 55.1M parameters, while D3PM-Small uses hidden dimension 512, 3 Transformer blocks, and 8 attention heads, yielding 15.9M parameters. Both variants are trained on the same derived training set and generate boards with 1000 denoising steps.

Table 7: Training configurations on NVIDIA RTX 4090 GPUs.

Task	Epochs	GPUs	Time
Sudoku-Extreme	50K	8	2h
ARC-AGI	200K	8	5 days
N-Queens (8×8)	3K	8	1h
N-Queens (10×10)	1K	8	3h
Graph Coloring (8 nodes)	5K	8	1.5h
Graph Coloring (10 nodes)	5K	8	6h
Binarized-MNIST	1.8K	8	16h
Sudoku-Generation	0.2K	4	0.6h

C Additional Details of Experiment Setup

C.1 Challenging Puzzle Tasks

ARC-AGI Evaluation Protocol. We follow the standard ARC-AGI evaluation protocol used by HRM [8] and TRM [9]. Each test input is transformed into 1,000 variants by combining rotations and reflections of the grid with permutations of its colors. The model is run on each variant, each prediction is mapped back by the inverse transformation, and the predictions are aggregated by voting; accuracy is reported as pass@2, the official ARC-AGI metric, under which a task counts as solved if either of the two submitted attempts is correct. For GRAM, we draw a single sample per augmented input, so the ARC-AGI numbers in Table 8 use neither parallel sampling nor trajectory selection. Augmentation is used only for ARC-AGI; all other benchmarks are evaluated on the original inputs. Appendix D.3 isolates the effect of parallel sampling on ARC-AGI by removing augmentation.

Looped TF on ARC-AGI. We report Looped Transformer [7] results on Sudoku-Extreme but omit them on ARC-AGI due to prohibitive training cost. Under the same setup used for the other recursive models on Sudoku-Extreme (batch size 768, on $8\times$ NVIDIA RTX 4090 GPUs), training Looped TF on Sudoku-Extreme already takes 19 hours. Measuring its per-step time directly on ARC-AGI and scaling to the 200K-epoch budget used for GRAM on ARC-AGI gives approximately **97 days** ($\approx$ 776 GPU-days) for a full training run.

This gap stems from two compounding factors. First, Looped TF lacks deep supervision: HRM, TRM, and GRAM perform N_{sup} gradient updates per trajectory (one per segment), whereas Looped TF performs only one update at the end of the full trajectory, slowing convergence. Second, Looped TF lacks adaptive halting such as ACT [8–10], so every input must be processed for the maximum recursion depth, increasing per-example sequential compute. Both inefficiencies compound at ARC-AGI scale, making a full Looped TF training run impractical.

C.2 Multi-solution Puzzle Tasks

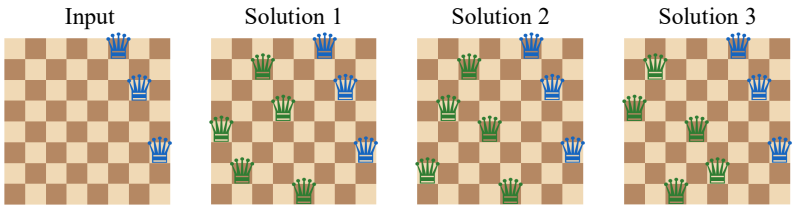


Figure 9: Example of an 8×8 N-Queens puzzle instance. In this example, 5 queens are removed from the full board, leaving 3 queens. The model must find the positions of the remaining queens. This configuration admits exactly 3 valid solutions.

N-Queens Problem. The N-Queens problem [58] requires placing N queens on an $N \times N$ chessboard such that no two queens attack each other, meaning no queens share the same row, column, or diagonal. Figure 9 illustrates an example where 5 queens are removed from an 8×8 solution, resulting in a puzzle with 3 distinct valid completions.

To construct the dataset, we first generated all valid complete N-Queens solutions for $N = 8$ and $N = 10$ by backtracking, yielding 92 and 724 solutions respectively. We then created puzzle instances by removing every combination of a fixed number of queens, treating the remaining partial configuration as the input and the original complete board as the target label. To generate instances yielding diverse valid completions, we removed $k \in \{5, 6, 7\}$ queens for the 8×8 setting and $k \in \{7, 8, 9\}$ queens for the 10×10 setting. Since a partial configuration can arise from several solutions, an input may carry multiple targets, on average 1.65 for 8×8 and 2.93 for 10×10. The distribution of solution counts for our generated dataset is shown in Figure 10.

For evaluation, we employed an 85:15 train-test split. Crucially, to prevent data leakage and ensure the model learns to reason rather than memorize, the split was performed based on unique *input* configurations. This guarantees that no input pattern in the test set appears in the training set. The resulting dataset contains 7,224 training and 1,240 test instances for 8×8, and 108,234 and 18,466 for 10×10. Inputs are flattened into discrete 1D sequences $x \in \{0, 1, 2\}^L$, where $L = N^2$, along with zero-padded puzzle embedding tokens. Vocabulary mapping follows: padding (0), empty (1), and queen (2). We use neither data augmentation nor rebalancing between single- and multi-solution instances.

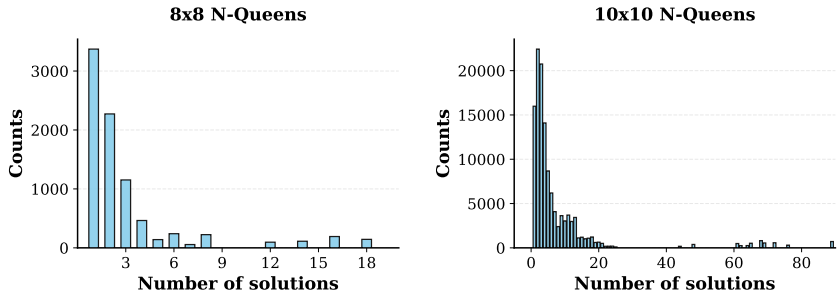


Figure 10: Distribution of the number of valid solutions for generated N-Queens instances. The dataset covers a wide range of solution counts, testing the model's ability to recover multiple valid outputs.

Graph Coloring Problem. The Graph Coloring problem requires assigning one of k colors to each node in a graph such that no two adjacent nodes share the same color. We consider graphs with

$N \in \{8, 10\}$ nodes and use $k = 3$ colors. Figure 11 illustrates an example instance with $N = 8$ nodes and $k = 3$ colors.

Graphs are generated using the Erdős–Rényi random graph model [59], following the generation pipeline from GNN-GCP [60], in which a CP-SAT solver verifies the chromatic number of each sampled graph. We retain only graphs whose chromatic number is exactly 3. The resulting graphs have 6 to 14 edges for $N = 8$ and 9 to 19 edges for $N = 10$. Mean edge density is 0.38 in training and 0.42 in test for $N = 8$, and 0.28 and 0.34 for $N = 10$.

For each graph, we enumerate all valid 3-colorings and retain only canonical forms, in which colors are relabelled in order of first appearance, to eliminate redundant solutions under color permutation. Since each graph admits several canonical colorings, a training graph may carry multiple targets, on average 3.40 for $N = 8$ and 8.73 for $N = 10$; the test split uses one coloring per graph. The distribution of solution counts is shown in Figure 12.

The final dataset consists of 7,002 training and 255 test instances for $N = 8$, and 13,465 training and 192 test instances for $N = 10$. The dataset is publicly available at https://huggingface.co/datasets/CUN-bjy/graph-coloring.

Input and Output Representation. The input graph is represented by extracting the upper triangular portion of the adjacency matrix (including the diagonal) and flattening it into a 1D sequence of $N(N+1)/2$ tokens. The output is a sequence of length N, where each position encodes the assigned color for the corresponding node, followed by PAD tokens up to the input length. Vocabulary mapping is as follows: PAD (0), no edge (1), edge (2), and colors (3, 4, 5) for red, blue, and green respectively.

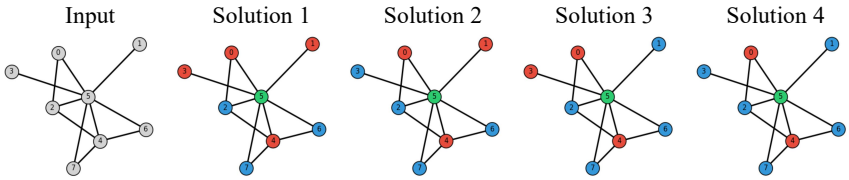


Figure 11: Graph Coloring Example

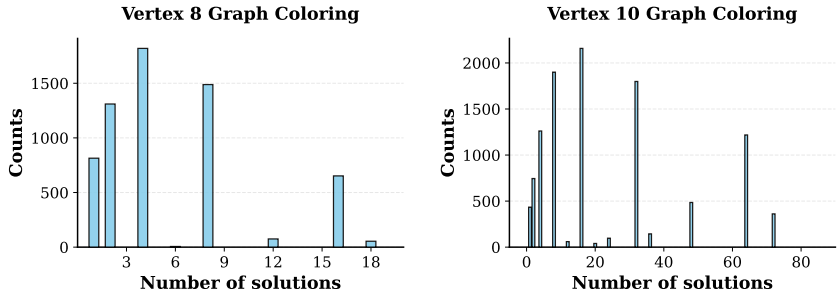


Figure 12: Distribution of the number of valid solutions for generated graph coloring instances. The dataset covers a wide range of solution counts, testing the model's ability to recover multiple valid outputs.

D Additional Experimental Results

D.1 Additional Results on Challenging Puzzle Benchmarks

Table 8 reports test accuracy on three challenging puzzle benchmarks. Here we provide additional observations complementing the main text.

GRAM Advances the Recursive-Reasoning Line. Across all three benchmarks, GRAM outperforms prior recursive baselines. On Sudoku-Extreme, GRAM already surpasses TRM with a single sample (86.6% against 83.6%), and parallel sampling, an axis the stochastic transition makes available and deterministic recursion does not have, raises it further to 96.1%. At matched inference

Table 8: Test accuracy (%) on Challenging Puzzle Benchmarks. All recursive model scores were obtained at 16 supervision steps; ARC-AGI accuracy is pass@2 following the standard protocol (Appendix C.1). Sudoku-Extreme scores of recursive models are averaged over 3 seeds and reported as mean ± standard deviation. ARC-AGI scores of HRM and TRM, all Direct Pred scores, and the Sudoku-Extreme scores of large reasoning models are taken from the original papers [8, 9], with ARC-AGI on the public evaluation set. ARC-AGI scores of large reasoning models are from the ARC Prize leaderboard [40] on the semi-private set. Bold marks the best result within each group.

Method	#Params	Sudoku	ARC-1	ARC-2
Large Reasoning Models				
Deepseek-R1	671B	0.0	15.8	1.3
Claude 3.7 16k	N/A	0.0	28.6	0.7
o3-mini-high	N/A	0.0	34.5	3.0
GPT 5.2 (low)	N/A	–	55.7	9.7
Grok-4-thinking	1.7T	–	66.7	16.0
Gemini 3 Pro	N/A	–	**75.0**	**31.1**
Recursive Models				
Direct Pred	27M	0.0	21.0	0.0
Looped TF	7M	$61.7_{\pm 0.9}$	-	-
HRM	27M	$56.8_{\pm 1.2}$	40.3	5.0
TRM	7M	$83.6_{\pm 1.3}$	44.6	7.8
GRAM (Ours), N=1	10M	$\mathbf{86.6}_{\pm 2.2}$	$\mathbf{50.3}_{\pm 2.3}$	$\mathbf{9.5}_{\pm 2.5}$
+ parallel sampling (N=20)		$\mathbf{96.1}_{\pm 0.9}$	–	–

compute, GRAM reaches 96.1% against TRM's 92.1% (Appendix D.2). The complete failure of direct prediction on Sudoku and ARC-AGI-2 (0% in both cases) further confirms that recursive computation is essential for these tasks — single-pass models, regardless of capacity, cannot solve them. Together, these results indicate that GRAM's gains arise from organizing recursive computation as probabilistic, multi-trajectory reasoning.

Sudoku-Extreme Resists Parameter Scaling. All tested large reasoning models (LRMs), including Deepseek-R1 (671B), score 0% on Sudoku-Extreme. This suggests that pretrained capacity alone does not transfer to constraint-propagation reasoning, and that benchmarks like Sudoku-Extreme probe a fundamentally different axis from those captured by general-purpose LRMs. On ARC-AGI, more recent LRMs such as Gemini 3 Pro (75.0% on ARC-1, 31.1% on ARC-2) remain substantially ahead of all recursive models, highlighting that abstract few-shot reasoning still benefits from scale; we view these numbers as benchmark-difficulty reference points rather than controlled baselines.

D.2 Computational Cost of Inference-Time Scaling

We measure the computational cost of inference-time scaling on Sudoku-Extreme. Floating-point operations are counted per puzzle with `torch.utils.flop_counter`, and wall-clock latency is measured on identical hardware with a single puzzle in flight, so that parallel samples are batched into one forward pass while recursion steps execute sequentially. Accuracies are the 3-seed means reported in Figure 4.

Stochastic transitions add little per-step cost. Table 9 reports inference FLOPs per puzzle. The stochastic transition introduces only the noise head, which produces the mean and variance of the latent perturbation, and leaves the recursive core unchanged. GRAM therefore costs 3.5% more per iteration than TRM.

Table 9: Inference FLOPs per puzzle in TFLOPs.

Model	Per iteration	16 iterations, N=1
Looped TF	0.0256	0.410
HRM	0.0138	0.220
TRM	0.0256	0.410
GRAM (Ours)	**0.0265**	**0.424**

At equal compute, width is more accurate and far faster than depth. Table 10 compares two configurations with nearly identical FLOPs: TRM run for 320 iterations, and GRAM run for 16 iterations with $N{=}20$ samples. Because the samples are independent, they execute in a single batched pass, whereas additional recursion iterations must run one after another. Spending compute on width therefore costs far less latency than spending it on depth, and it also yields higher accuracy.

Table 10: Matched-FLOPs comparison on Sudoku-Extreme. Latency is measured per puzzle with a single request in flight.

Configuration	TFLOPs / puzzle	Latency (ms)	Accuracy
TRM, 320 iterations, $N{=}1$	8.19	2,755	92.15
GRAM, 16 iterations, $N{=}20$	8.48	**184**	**96.06**
Relative	$+3.5\%$	$\mathbf{15.0\times}$ **faster**	$\mathbf{+3.9}$ **pp**

GRAM reaches a given accuracy with less compute. Table 11 reports the compute at which each model first reaches a target accuracy, obtained by sweeping the number of recursion iterations and interpolating log-linearly between measured points; we report targets that both models reach within the measured range.

Table 11: Inference compute in TFLOPs per puzzle required to reach a target accuracy on Sudoku-Extreme.

Target accuracy	TRM	GRAM ($N{=}1$)	Savings
85	0.566	0.357	$1.6\times$
88	1.368	0.536	$2.6\times$
90	2.802	0.750	$\mathbf{3.7}\times$

Numerical values behind Figure 4. Table 12 reports the accuracy for every configuration shown in Figure 4. The corresponding inference compute is the product of the number of iterations, the per-iteration cost in Table 9, and the number of samples N.

Table 12: Accuracy (%) on Sudoku-Extreme as a function of recursion iterations, with the corresponding inference cost. Mean $\pm$ standard deviation over 3 seeds. The compute for any entry is the number of iterations times the per-iteration cost, which for GRAM includes all N samples.

Model	TFLOPs / iter	8	16	32	128	320
Looped TF	0.0256	$50.0{\pm}2.5$	$61.7{\pm}0.9$	$64.5{\pm}0.9$	$65.8{\pm}1.0$	$66.1{\pm}1.0$
HRM	0.0138	$50.4{\pm}1.1$	$56.8{\pm}1.2$	$60.2{\pm}1.3$	$63.8{\pm}1.5$	$65.4{\pm}1.6$
TRM	0.0256	$79.3{\pm}1.2$	$83.6{\pm}1.3$	$86.6{\pm}1.4$	$90.4{\pm}1.5$	$92.1{\pm}1.4$
GRAM, $N{=}1$	0.0265	$80.2{\pm}2.4$	$86.6{\pm}2.2$	$90.7{\pm}1.8$	–	–
GRAM, $N{=}5$	0.1325	$89.8{\pm}2.0$	$93.6{\pm}1.4$	$95.3{\pm}1.1$	–	–
GRAM, $N{=}10$	0.2650	$92.4{\pm}1.6$	$95.1{\pm}1.1$	$96.3{\pm}0.8$	–	–
GRAM, $N{=}20$	0.5300	$94.1{\pm}1.3$	$96.1{\pm}0.9$	$96.9{\pm}0.7$	–	–
GRAM, $N{=}50$	1.3250	$95.4{\pm}1.0$	$96.8{\pm}0.7$	$97.4{\pm}0.6$	–	–

D.3 Scaling with Parallel Sampling on ARC-AGI Challenge

To investigate the effect of GRAM's sampling on the ARC-AGI-1 benchmark, we measured performance without test-time augmentation. Typically, TRM achieves its reported accuracy by generating 1,000 augmentations for a single problem and performing majority voting over the results. Because this augmentation process itself creates a wide variety of samples, we isolated the specific effect of generative sampling by performing inference solely on the original problem instance and conducting majority voting over multiple sampled paths. For a fair comparison, we reproduced TRM and evaluated it under the same setting as GRAM, on the original problem instances without test-time augmentation.

As illustrated in Figure 13, without test-time augmentation, both GRAM and TRM fall below the values reported in Table 8. However, in the case of GRAM, we observe that accuracy consistently

improves as the model generates more parallel samples. This trend mirrors observations in Section 4.1, suggesting that increased inference-time compute through width scaling allows the model to explore more plausible reasoning trajectories and recover from initial errors, eventually leading to more robust solution discovery.

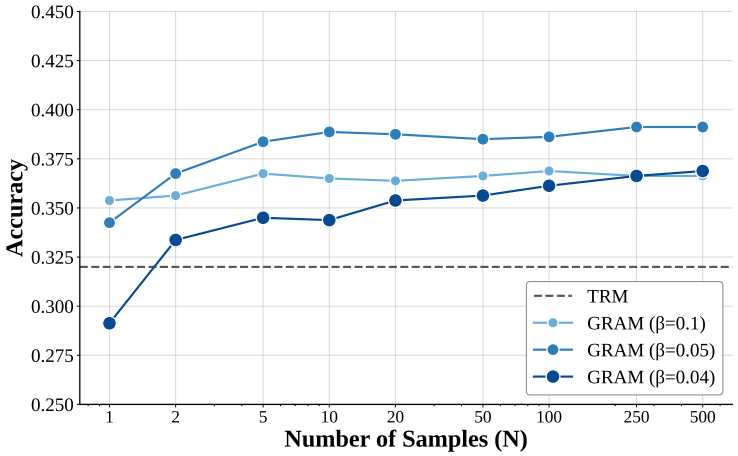


Figure 13: Effect of sampling on ARC-AGI-1 without test-time augmentation. To isolate the internal sampling effect, GRAM and TRM are evaluated on original problem instances without 1,000 augmentations. GRAM is shown at three KL coefficients β. While removing augmentations causes an initial performance drop, GRAM scales consistently with the number of parallel samples N at every β.

Interaction between Augmentation and Sampling. A natural question arises: why not combine higher levels of augmentation with extensive parallel sampling? To address this, we conducted an ablation study examining the interaction between test-time augmentation and parallel sampling. Figure 14 presents the results for GRAM with $\beta = 0.05$ across varying augmentation levels (Aug=0 to Aug=50). Without augmentation (Aug=0), increasing the number of samples yields consistent accuracy improvements, demonstrating that stochastic sampling effectively explores diverse reasoning trajectories. However, as the level of augmentation increases, the marginal benefit of additional sampling diminishes substantially. At Aug=50, performance saturates regardless of sample count—accuracy remains nearly constant whether we draw 1 or 50 samples. This observation reveals that augmentation and sampling are not additive, because both produce multiple predictions per input and aggregate them by voting. Augmentation obtains these predictions from hand-designed, task-specific transformations of the input, while sampling obtains them from the stochastic transitions of a single model without such design. When test-time augmentation is limited, parallel sampling compensates by exploring varied reasoning paths at inference time. When test-time augmentation is abundant, the aggregated predictions over augmented inputs already provide sufficient diversity, rendering additional inference-time exploration redundant. Consequently, scaling sampling beyond augmentation provides diminishing returns, justifying our experimental design choice to evaluate these two scaling axes separately.

D.4 Analysis of Model Performance by Number of Valid Solutions

We group N-Queens test instances by the number of valid solutions each input admits and report single-sample accuracy and solution coverage for each group.

Accuracy. Figure 15 shows single-sample accuracy on 8×8 N-Queens. All three deterministic recursive baselines degrade steadily as the number of valid solutions per input grows, whereas GRAM remains flat across the entire range. Since this comparison uses a single trajectory for GRAM, the gap does not come from parallel sampling. It instead reflects the training signal: inputs with more valid solutions carry more conflicting targets, which a deterministic model must collapse into one output, whereas GRAM absorbs the ambiguity in its latent variable.

Coverage. Figure 16 shows solution coverage on 8×8 and 10×10 N-Queens. Deterministic recursive models (HRM and TRM) decline sharply as the number of valid solutions increases, since a

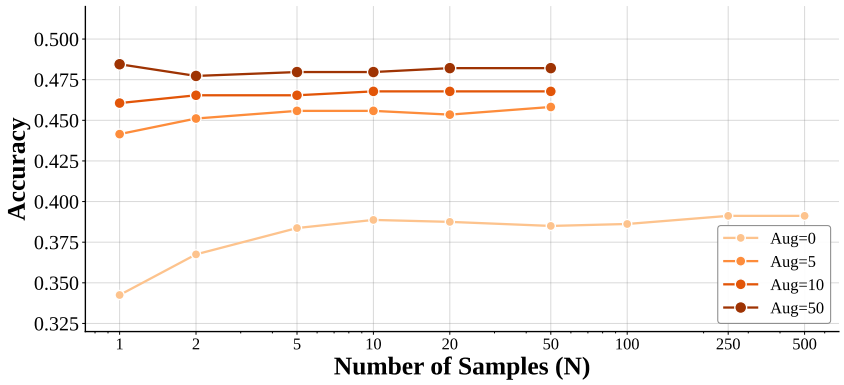


Figure 14: Effect of augmentation on sampling efficiency. With limited augmentation (Aug=0), parallel sampling provides consistent gains. As augmentation increases, sampling benefits diminish—at Aug$=50$, performance saturates regardless of sample count, suggesting that both provide multiple predictions for voting, and sampling does so without task-specific transformations.

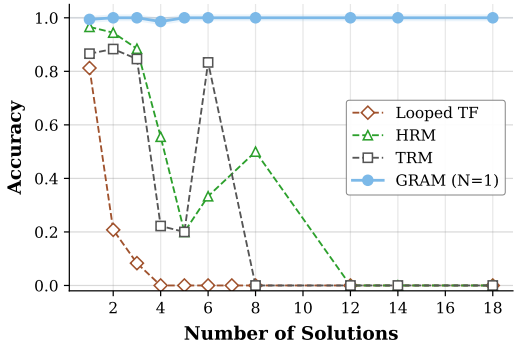


Figure 15: Accuracy across the number of valid solutions in N-Queens (8×8). Conventional deterministic recursive models suffer a sharp performance drop as the number of possible solutions increases, whereas GRAM maintains consistent performance with a single sample ($N = 1$).

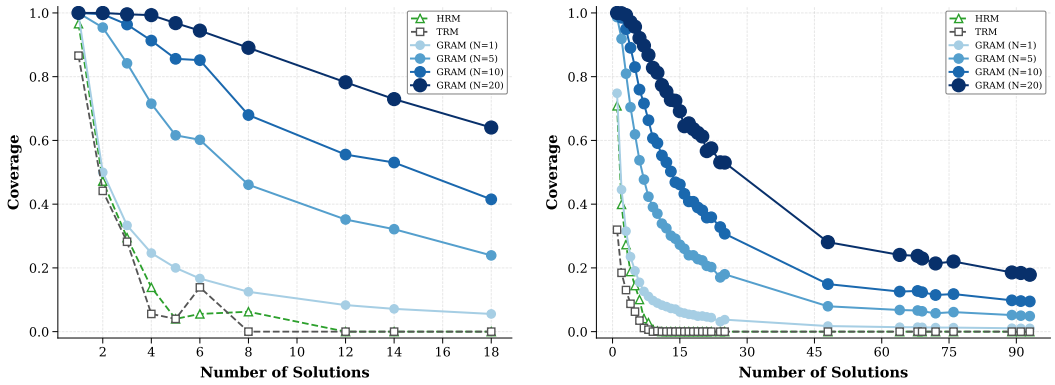
single fixed reasoning trajectory cannot reach alternative solutions no matter how long it is refined. GRAM instead improves monotonically as the number of parallel samples N grows from 1 to 20, in both settings. Recovering multiple solutions therefore requires sampling from a distribution over trajectories rather than refining a single one, with Table 3b isolating the contribution of stochastic guidance itself.

D.5 Additional Generated Samples

GRAM samples and progressive refinement. In this section, we provide further qualitative results demonstrating GRAM's capability in unconditional image generation. Figure 17 presents a diverse set of samples generated on the binarized MNIST dataset, visualized across the recursive inference steps $t = 0$ to $t = 16$.

As observed in the main text, GRAM exhibits a distinct progressive refinement behavior. Starting from a black initialization, the model iteratively adds details and sharpens the structure of the digit. A particularly compelling property of this process is the model's ability to recover from initially ambiguous or incorrect formations.

For instance, in the second row (generating the digit '2') and the last row (generating the digit '1'), the early predictions at $t = 1$ and $t = 2$ manifest as disjointed artifacts or incorrect shapes. However, as the recursion proceeds, GRAM effectively leverages its feedback loop to correct these initial errors, resolving the ambiguity and converging to a coherent, high-quality digit by $t = 16$.



(a) N-Queens 8×8 **(b)** N-Queens 10×10

Figure 16: Solution coverage analysis on N-Queens (8×8 and 10×10) with respect to the number of ground-truth solutions. While deterministic baselines (HRM, TRM) suffer from mode collapse as the solution space grows, GRAM demonstrates monotonic improvement in coverage as the number of parallel samples N increases.

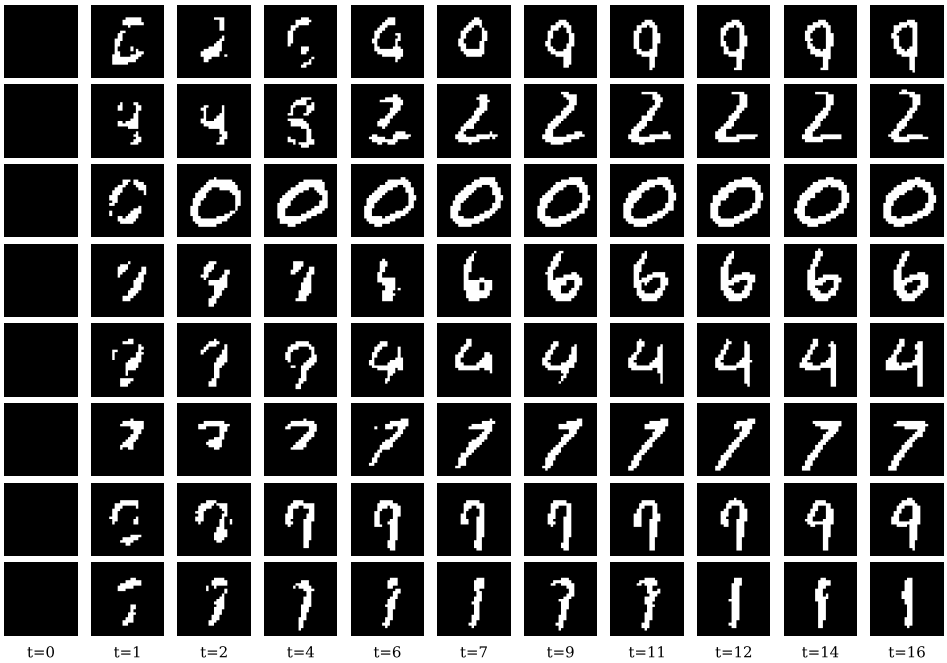


Figure 17: Additional generated samples from GRAM. We provide 8 additional samples generated unconditionally on binarized MNIST using GRAM. Each row represents a single generated sample, visualized across its recursive refinement process.

VAE baseline samples. For reference, Figure 18 shows unconditional samples from the VAE baseline reported in Table 2. These samples are generated by the BCE loss variant used in the main comparison to maintain a consistent discrete-pixel representation across models.

D.6 Additional Results on Unconditional Sudoku Generation

As shown in Table 13, GRAM achieves 99.05% validity, outperforming all D3PM baselines. The D3PM (Big) baseline reaches 79.18% validity while using 55.1M parameters and 1000 denoising steps. In contrast, GRAM uses fewer parameters and only 16 inference steps. In all cases, the valid samples are unique under exact board matching, indicating that the reported validity is not due to simple repetition of a small set of boards. These results show that GRAM can generate highly

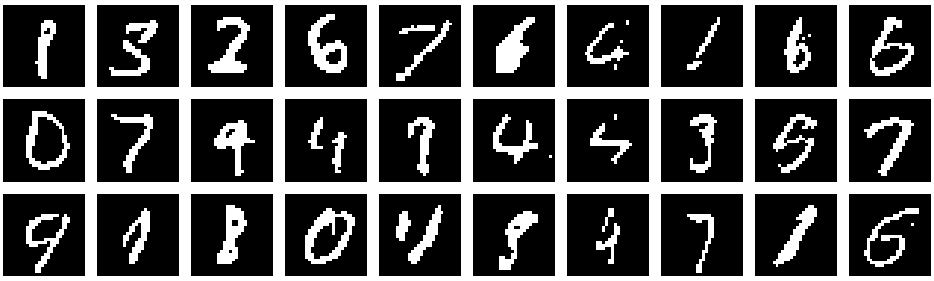


Figure 18: Unconditional samples from the VAE baseline. Samples generated by the VAE reported in Table 2.

constrained symbolic structures from an empty input, supporting its potential as a generator beyond conditional puzzle solving.

Table 13: Unconditional Sudoku generation. We report the ratio of generated boards satisfying Sudoku constraints over 100K samples. All valid boards are unique for all methods in this evaluation.

Method	#Params	Steps	Validity(%)
D3PM (Big)	55.1M	1000	79.18
D3PM (Small)	15.9M	1000	21.88
GRAM (Ours)	**10.9M**	**16**	**99.05**

D.7 Stochasticity and Guidance Ablation without Parallel Sampling

Table 3b in the main text reports the stochasticity and guidance ablation at N=20. Table 14 adds the single-sample results, so that the contribution of stochasticity and learned guidance can be read separately from the effect of parallel sampling. TRM with stochastic decoding samples only the output tokens, and on Sudoku its output distribution is nearly one-hot, so its samples match the argmax prediction and its two Sudoku columns coincide.

Table 14: Stochasticity and guidance ablation at N=1 and N=20 on Sudoku-Extreme and N-Queens (8×8). Values are means over 3 seeds with standard deviations.

	Sudoku		N-Queens	
Model variant	N=1	N=20	N=1	N=20
GRAM (ours)	$86.6_{\pm 2.2}$	$\mathbf{96.1}_{\pm 0.9}$	$99.5_{\pm 0.3}$	$\mathbf{100.0}_{\pm 0.0}$
w/o stochastic guidance	$83.6_{\pm 1.3}$	$83.6_{\pm 1.3}$	$66.8_{\pm 5.6}$	$66.8_{\pm 5.6}$
stochasticity only	$86.6_{\pm 1.4}$	$96.0_{\pm 1.1}$	$60.2_{\pm 15.1}$	$62.1_{\pm 14.9}$
guide only	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$
w/ direct prediction	$44.1_{\pm 5.2}$	$70.7_{\pm 7.6}$	$97.5_{\pm 3.5}$	$98.8_{\pm 2.2}$
TRM w/ stochastic decoder	$79.7_{\pm 1.9}$	$79.7_{\pm 1.9}$	$56.8_{\pm 5.7}$	$65.9_{\pm 4.7}$
TRM w/ random init.	$79.9_{\pm 1.2}$	$91.2_{\pm 0.9}$	$71.1_{\pm 8.5}$	$71.4_{\pm 8.5}$

D.8 Placement of the Stochastic Transition

The hierarchical instantiation admits a stochastic transition at either level, the high-level update or the low-level refinement. We compare three low-level variants with our high-level design on N-Queens 8×8. *Every-low* samples ϵ at all K low-level refinements, trained either with the KL of the final refinement only (Last KL) or with the KL averaged over the K refinements (Mean KL). *Last-low* samples ϵ once, at the final low-level refinement, and uses only that KL term.

Table 15 shows that Every-low fails under both KL variants, while Last-low matches our high-level design. This suggests that the number of injections per transition matters more than the level at which they occur: Every-low draws from the posterior K times within a single high-level update

Table 15: Placement of the stochastic transition in GRAM on N-Queens 8×8. Accuracy is single-sample and coverage uses 20 samples.

Placement	Accuracy	Coverage
High-level (default)	99.5 ± 0.3	89.7 ± 1.7
Every-low, Last KL	0.0 ± 0.0	0.0 ± 0.0
Every-low, Mean KL	0.0 ± 0.0	0.0 ± 0.0
Last-low, Last KL	99.2 ± 0.6	90.2 ± 0.4

(K=4 here), so each transition receives K times as much target information, which we attribute to the resulting overfitting. Our formulation defines the stochastic transition on z_t and is agnostic to placement, so we keep the injection at the high level, which trains stably.

D.9 Reinforcement Learning Fine-Tuning of GRAM

We investigate whether GRAM can be further improved with outcome-level feedback by updating its stochastic transition prior using proximal policy optimization (PPO) [61], while freezing the remaining pretrained parameters.

Setup. At each recursive step, we treat the stochastic residual $\epsilon_t \sim p_\theta(\epsilon_t \mid u_t)$ as the policy action that determines the next latent transition. After each transition, we assign a binary reward of one if the current prediction is a valid N-Queens solution and zero otherwise. A trajectory terminates upon finding a valid solution or after 16 steps. We train for 1,000 PPO updates with a batch of 512 trajectories, using eight trajectories per problem. We use $\gamma = 0.99$, GAE [62] $\lambda = 0.95$, a clipping ratio of 0.2, two PPO epochs, and learning rates of 3×10^{-5} and 10^{-4} for the transition prior and PPO critic, respectively.

Results. Table 16 reports single-sample accuracy (N=1), majority-vote accuracy with 20 samples (N=20), and coverage using the same 20 samples. On 8×8 N-Queens, GRAM is already near saturation, and reinforcement learning provides modest additional gains. On 10×10 N-Queens, GRAM-RL improves single-sample accuracy from 89.7% to 99.1% and coverage from 57.5% to 61.8%, while majority-vote accuracy remains saturated.

Table 16: Reinforcement learning fine-tuning on N-Queens. We report single-sample accuracy (N=1), majority-vote accuracy with 20 samples (N=20), and coverage with 20 samples. Values are means ± sample standard deviations over 3 seeds.

	N-Queens (8×8)			**N-Queens** (10×10)		
Method	N=1	N=20	Coverage	N=1	N=20	Coverage
GRAM (Ours)	99.5±0.3	100.0±0.0	89.7±1.7	89.7±2.7	100.0±0.0	57.5±3.4
GRAM-RL	**99.9**±0.2	100.0±0.0	**90.7**±0.5	**99.1**±0.3	100.0±0.0	**61.8**±0.2

D.10 Visualizing Latent Recursion Process

To understand how stochastic guidance shapes reasoning, we visualize latent trajectories during recursive computation. Specifically, we track the high-level state h at each supervision step throughout the recursion process. For visualization, we project these latent vectors into 2D using PCA [63] and interpolate unobserved states via K-D tree [64] to construct a continuous loss landscape.

Figures 19 and 20 compare TRM and GRAM on the same Sudoku puzzle. TRM follows a single deterministic path from initialization to solution, offering no mechanism to escape if the trajectory enters a suboptimal region. In contrast, GRAM samples diverse trajectories that explore different regions of latent space before converging. While some trajectories become trapped in local minima (bottom right), others successfully navigate toward the global optimum (dark blue regions). This diversity enables GRAM to discover valid solutions more reliably through parallel exploration.

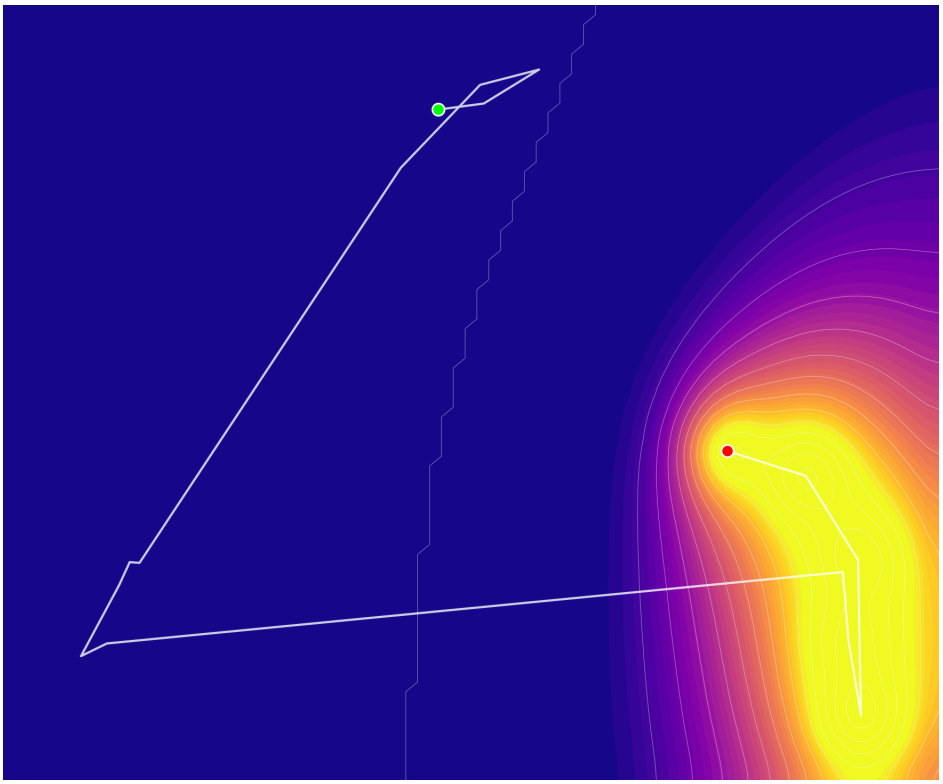


Figure 19: Latent reasoning trajectory of TRM. The red dot indicates the initial state h_0 and the green dot indicates the final state h_T. Background color represents the loss landscape: bright yellow corresponds to high loss regions, while dark blue indicates low loss (optimal) regions. TRM follows a single deterministic path with no ability to escape suboptimal trajectories.

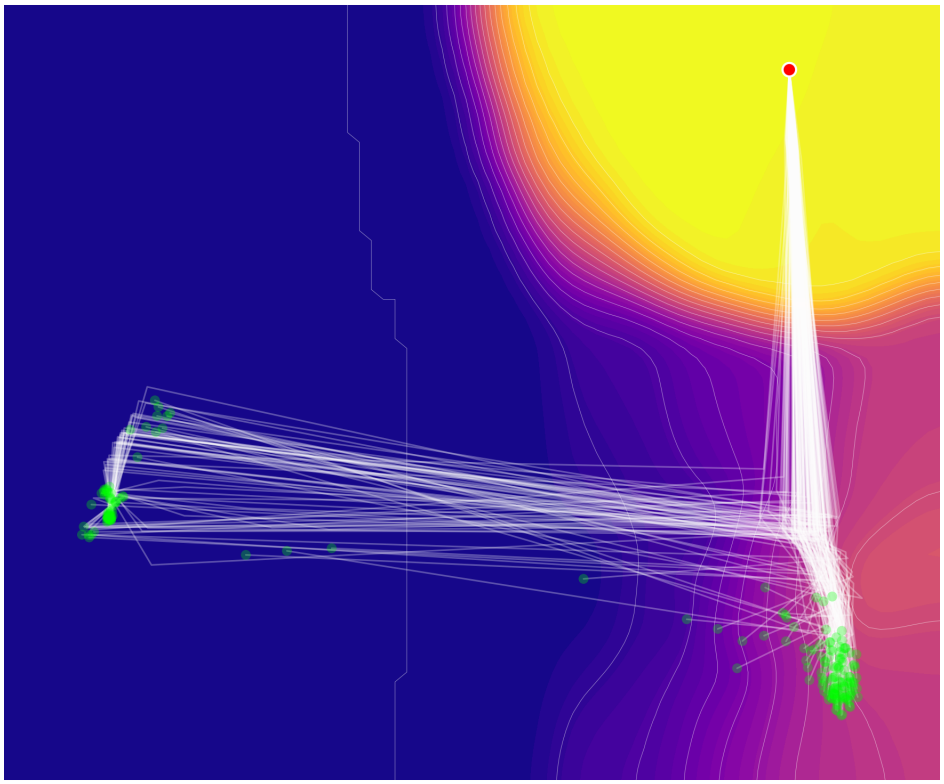


Figure 20: Latent reasoning trajectories of GRAM (50 samples). Using the same visualization scheme as Figure 19, we show 50 sampled trajectories from GRAM. The stochastic guidance enables diverse exploration of the latent space: while some trajectories converge to local minima (right bottom), others successfully reach the global optimum (left middle), demonstrating how parallel sampling improves solution discovery.

Licenses

Table 17: Existing assets, licenses, and source links. We list the existing datasets, benchmarks, and public reference implementations used or cited in our experiments. Synthetic N-Queens and Graph Coloring instances are generated by the authors and are therefore not external assets.

Asset	Use in this paper	License / terms	Source link
MNIST	Binarized MNIST generation experiments	Creative Commons Attribution-Share Alike 3.0	https://keras.io/api/datasets/mnist/
ARC-AGI-1 / original ARC	ARC-AGI reasoning benchmark	Apache License 2.0	https://github.com/fchollet/ARC-AGI
ARC-AGI-2	ARC-AGI-2 reasoning benchmark / reference results	Apache License 2.0	https://github.com/arcprize/ARC-AGI-2
HRM repository	HRM baseline and Sudoku-Extreme-related reference implementation	Apache License 2.0	https://github.com/sapientinc/HRM
TinyRecursiveModels / TRM repository	TRM baseline and recursive reasoning reference implementation	MIT License	https://github.com/SamsungSAILMontreal/TinyRecursiveModels
MDLM repository	Masked diffusion baseline reference implementation	Apache License 2.0	https://github.com/kuleshov-group/mdlm
Google Research D3PM implementation	D3PM image-generation baseline reference implementation	Apache License 2.0	https://github.com/google-research/google-research/blob/master/d3pm/images/diffusion_categorical.py
Looped Transformer repository	Looped Transformer baseline reference implementation	MIT License	https://github.com/Leiay/looped_transformer
N-Queens	Synthetic multi-solution constraint satisfaction task generated by the authors	Not an external asset	N/A
Graph Coloring	Synthetic multi-solution constraint satisfaction task generated by the authors	Not an external asset	N/A